

Pipeline Escalar e Superescalar

Daniel Lucas Soares Madureira¹, Gabriel Marques da Cunha¹, Vinicius Cezar Pereira Menezes¹

¹ Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
ICEI – Instituto de Ciências Exatas e de Informática
Engenharia de Computação

{1433380, gabriel.cunha.1296091, vcpmenezes}@sga.pucminas.br

Resumo. *Este trabalho analisa o paralelismo em nível de instrução por meio do estudo de pipelines escalares e superescalares. Inicialmente, são investigados hazards de dados e controle em um pipeline escalar de 5 estágios, avaliando técnicas como forwarding e predição de desvios. Em seguida, aborda-se a arquitetura superescalar, com foco em despacho múltiplo, execução fora de ordem e aplicação do Algoritmo de Tomasulo. A renomeação de registradores é utilizada para eliminar falsas dependências e ampliar o paralelismo. Por fim, os resultados são integrados por meio da análise de métricas como CPI e IPC, discutindo limites, custos e mecanismos de hardware voltados à melhoria de desempenho em processadores modernos.*

1. Análise do Pipeline: Sequência S2 (Loop com Dependência de Variável de Indução)

Nesta seção, analisa-se o comportamento do pipeline escalar de 5 estágios (*IF*, *ID*, *EX*, *MEM*, *WB*) durante a execução das iterações iniciais da Sequência S2. O objetivo é demonstrar o impacto das dependências de dados (*Hazards RAW*) e de controle no fluxo de instruções, comparando arquiteturas com e sem a técnica de *forwarding* (encaminhamento de dados).

Por se tratar de um loop contínuo, foi mapeado o comportamento da primeira iteração e da transição para a segunda, visto que o padrão de bolhas (*stalls*) e descartes (*flushes*) se repete ao longo da execução.

1.1 Cenário A: Execução Sem *Forwarding*

Na ausência de um mecanismo de repasse interno de dados, qualquer instrução que dependa de um valor recém-calculado deve aguardar passivamente até que a instrução geradora grave esse dado no Banco de Registradores (estágio *WB*). Assumindo que a gravação ocorre na primeira metade do ciclo e a leitura na segunda metade, temos o seguinte diagrama de espaço-tempo:

Tabela 1. Diagrama de espaço-tempo – Cenário A (Sem Forwarding)

Instrução (Iteração 1)	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14
lw x13, 0(x12)	IF	ID	EX	MEM	WB									
add x10, x10, x13		IF	ID	–	–	EX	MEM	WB						
addi x12, x12, 4			IF	–	–	ID	EX	MEM	WB					
addi x11, x11, -1						IF	ID	EX	MEM	WB				
bne x11, zero, loop							IF	ID	–	–	EX	MEM	WB	
(Instruções erradas)								IF	–	–	ID	(flush)		
(Iteração 2) lw x13												IF	ID	EX

Análise de Stalls (Sem Forwarding)

Neste cenário, observamos a ocorrência de **4 bolhas (stalls) por iteração** geradas puramente por dependências de dados (RAW):

- **Load-Use (lw → add):** A instrução add fica travada no estágio *ID* durante os ciclos 4 e 5, aguardando que o lw alcance o estágio *WB* para liberar o registrador x13. Isso gera 2 ciclos de *stall*.
- **Dependência do Branch (addi → bne):** O desvio bne depende do cálculo de x11 feito pelo addi. O bne trava no *ID* nos ciclos 9 e 10, aguardando a gravação no *WB* no ciclo 10, gerando mais 2 ciclos de *stall*.

Além disso, assumindo a falta de predição de saltos, há a **penalidade de controle**. O pipeline busca instruções incorretas até que o desvio seja resolvido, resultando no descarte (*flush*) dessas instruções e atrasando a busca da próxima iteração (novo lw) para o ciclo 12.

1.2 Cenário B: Execução Com Forwarding

Com a introdução do *forwarding*, o processador consegue repassar os dados diretamente da saída da ALU ou da Memória de Dados para a entrada da ALU da instrução seguinte, mitigando grande parte dos atrasos. No entanto, dependências específicas ainda exigem paradas no pipeline.

Tabela 2. Diagrama de espaço-tempo – Cenário B (Com Forwarding)

Instrução (Iteração 1)	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14
lw x13, 0(x12)	IF	ID	EX	MEM	WB									
add x10, x10, x13		IF	ID	–	EX	MEM	WB							
addi x12, x12, 4			IF	–	ID	EX	MEM	WB						
addi x11, x11, -1					IF	ID	EX	MEM	WB					
bne x11, zero, loop						IF	ID	EX	MEM	WB		MEM	WB	
(Instrução errada 1)							IF	ID	(flush)			(flush)		
(Instrução errada 2)							IF	(flush)				IF	ID	EX

Análise de Stalls (Com Forwarding)

A implementação do repasse de dados elimina os travamentos do bne, mas evidencia um **hazard** estrutural/dados irreduzível. Temos **1 bolha (stall) por iteração**:

- **Load-Use Irreduzível (lw → add):** O valor de x13 lido pela instrução lw só está efetivamente disponível ao final do seu estágio *MEM* (ciclo 4). Como a instrução add necessita deste mesmo valor no início do seu estágio *EX*, é fisicamente

impossível repassar o dado a tempo. Consequentemente, a arquitetura é forçada a inserir 1 ciclo de *stall* no ciclo 4.

A penalidade de controle permanece presente nesta configuração básica (sem preditor dinâmico), ocorrendo o *flush* no momento em que o desvio é tomado, reiniciando o fluxo correto no ciclo 9.

2. Escalonamento Estático (Sequência S2)

O escalonamento estático é uma técnica de otimização em nível de compilador (ou programador) que reordena as instruções de um código para evitar bolhas (*stalls*) no pipeline, mantendo a semântica original do programa.

Na sequência original da S2, identificamos um *stall* de 1 ciclo gerado pela dependência de dados (*RAW*) entre a leitura da memória (*lw*) e a soma (*add*).

Código Original (com *Stall*):

```
1 loop:
2     lw    x13, 0(x12)      # Le o dado da memoria
3     add   x10, x10, x13    # STALL: x13 ainda nao disponivel
4     addi  x12, x12, 4      # Incrementa o ponteiro
5     addi  x11, x11, -1     # Decrementa o contador
6     bne   x11, zero, loop  # Desvio
```

Listing 1. Código Original (com *Stall*)

```
1 loop:
2     lw    x13, 0(x12)      # Le o dado da memoria
3     addi  x12, x12, 4      # Preenche o ciclo de latencia do Load-
                          Use
4     add   x10, x10, x13    # x13 ja esta pronto via forwarding
5     addi  x11, x11, -1     # Decrementa o contador
6     bne   x11, zero, loop  # Desvio
```

Listing 2. Código Reordenado (Otimizado)

Análise da Preservação Semântica

A semântica do programa é totalmente preservada. A instrução *addi x12, x12, 4* altera apenas o registrador *x12* (ponteiro do array). O *lw* já utilizou o valor antigo de *x12* no momento da busca, e o *add* depende apenas de *x13* e *x10*. Portanto, não há conflito de registradores. Ao afastar o *add* em um ciclo cronológico em relação ao *lw*, o repasse de dados (*forwarding*) do final do estágio *MEM* do *lw* para o início do estágio *EX* do *add* torna-se fisicamente viável, reduzindo o CPI do loop.

3. Prova Analítica do Problema de Load-Use (Sequência S3)

O *Hazard* de Load-Use ocorre quando uma instrução de leitura de memória (*Load*) é imediatamente seguida por uma instrução que utiliza o registrador de destino dessa leitura.

Para provar analiticamente a impossibilidade de resolver essa dependência apenas com *forwarding*, devemos analisar a linha do tempo dos estágios do pipeline:

1. **Instrução 1 (Load):** O endereço efetivo da memória é calculado no estágio de Execução (*EX*). O acesso real à Memória de Dados ocorre apenas no estágio de Memória (*MEM*). O dado lido (*D*) só fica fisicamente disponível na saída da memória ao final do estágio *MEM*.
2. **Instrução 2 (Instrução dependente):** Se emitida imediatamente após o *Load*, esta instrução estará no seu estágio de Execução (*EX*) no exato mesmo ciclo de clock em que o *Load* está no estágio *MEM*. A instrução dependente precisa do dado (*D*) na entrada da sua ALU no início do seu estágio *EX*.

Conclusão Analítica: Para que o *forwarding* resolvesse o conflito sem interromper o fluxo, o dado precisaria ser repassado do final do estágio *MEM* da primeira instrução para o início do estágio *EX* da segunda instrução, que ocorre no mesmo ciclo de clock. Como é fisicamente impossível repassar uma informação “para o passado” ou para o início de um ciclo que já está ocorrendo, a arquitetura é forçada a congelar o pipeline.

A inserção de 1 ciclo de bolha (*stall*) atrasa o estágio *EX* da instrução dependente para o próximo ciclo de clock. Somente com esse atraso estrutural, o *forwarding* consegue encaminhar o dado da saída do registrador de pipeline (*MEM/WB*) da instrução de *Load* para a entrada da ALU da instrução dependente.

4. Simulação do Preditor de Desvio de 2 Bits (Sequência S5)

Nesta seção, construímos a tabela de transição de estados do preditor dinâmico de 2 bits para os primeiros 8 desvios da sequência analisada. O autômato de predição (máquina de estados finitos) utiliza 4 estados baseados em histerese para a tomada de decisão:

- **SNT** (*Strongly Not Taken*): Fortemente Não Tomado (Prediz: Não Tomado)
- **WNT** (*Weakly Not Taken*): Fracamente Não Tomado (Prediz: Não Tomado)
- **WT** (*Weakly Taken*): Fracamente Tomado (Prediz: Tomado)
- **ST** (*Strongly Taken*): Fortemente Tomado (Prediz: Tomado)

Para esta simulação manual, consideramos a execução de um laço de repetição com 8 avaliações de desvio. O comportamento real (*ground truth*) do programa consiste em saltar de volta para o início do loop nas 7 primeiras vezes (Tomado/*Taken* – T) e sair do loop na oitava vez (Não Tomado/*Not-Taken* – N). O estado inicial adotado para a simulação a frio (*cold start*) é o SNT.

Tabela 3. Tabela de transição de estados – Preditor de 2 bits

Iteração	Estado Atual	Predição do Hardware	Resultado Real	Diagnóstico	Próximo Estado
1	SNT	Não Tomado (N)	Tomado (T)	Erro	WNT
2	WNT	Não Tomado (N)	Tomado (T)	Erro	WT
3	WT	Tomado (T)	Tomado (T)	Acerto	ST
4	ST	Tomado (T)	Tomado (T)	Acerto	ST
5	ST	Tomado (T)	Tomado (T)	Acerto	ST
6	ST	Tomado (T)	Tomado (T)	Acerto	ST
7	ST	Tomado (T)	Tomado (T)	Acerto	ST
8	ST	Tomado (T)	Não Tomado (N)	Erro	WT

Análise de Desempenho da Predição

- **Total de Acertos: 5**

- **Total de Erros:** 3
- **Taxa de Acerto Inicial:** 62,5% (5/8)

Conclusão sobre o Comportamento do Preditor: Observamos que o preditor de 2 bits comete dois erros iniciais consecutivos. Isso ocorre devido ao seu período de aquecimento (*warm-up*), no qual os contadores estão transitando do estado fortemente negativo (SNT) até alcançarem o limiar de predição positiva (WT e ST). A partir da terceira iteração, o histórico absorve o padrão repetitivo e o hardware passa a prever corretamente todas as iterações internas do loop. O terceiro erro ocorre exclusivamente na quebra do padrão, ou seja, na última iteração, quando a condição de saída do loop é finalmente atingida.

Vantagem Teórica: A superioridade arquitetural do preditor de 2 bits (em comparação ao de 1 bit) revela-se caso esse mesmo trecho de código seja executado novamente (como em loops aninhados). Ao fim das 8 iterações, o autômato estacionou no estado WT. Se o loop recomeçar imediatamente, a primeira predição já será “Tomado” (T). Dessa forma, a inércia dos 2 bits protege o histórico, eliminando os erros de aquecimento nas execuções subsequentes e elevando drasticamente a taxa de acerto global.

4.1 Análise da Sequência S1 — Cadeia RAW

Resultado das Simulações

A Tabela 4 apresenta os resultados das simulações da Sequência S1 sob diferentes configurações de pipeline escalar.

Tabela 4. Resultados das simulações — Sequência S1

Configuração	Instruções	Ciclos	Stalls / Flushes	CPI
Sem <i>forwarding</i> , sem predição	5	9	0 / 0	1,80
Com <i>forwarding</i> , sem predição	9	15	2 / 0	1,67
Com <i>forwarding</i> + predição estática	9	15	3 / 0	1,67
Com <i>forwarding</i> + predição dinâmica 1-bit	9	15	4 / 0	1,67
Com <i>forwarding</i> + predição dinâmica 2-bits	9	15	5 / 0	1,67

Q1 — Pipeline Sem *Forwarding*

Neste cenário, uma instrução só pode ler um registrador no estágio *ID* após a instrução geradora tê-lo gravado no banco de registradores no estágio *WB*. O diagrama abaixo ilustra a propagação das bolhas (*stalls*) causadas por essa restrição física nas dependências de distância 1.

Tabela 5. Diagrama de espaço-tempo — S1 sem *forwarding*

Instrução	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14
I1: <i>addi</i> t0, zero, 10	IF	ID	EX	MEM	WB									
I2: <i>add</i> t1, t0, t0		IF	ID	–	–	EX	MEM	WB						
I3: <i>add</i> t2, t1, t0			IF	–	–	ID	–	–	EX	MEM	WB			
I4: <i>add</i> t3, t2, t1						IF	–	–	ID	–	–	EX	MEM	WB

Os ciclos que contêm bolhas são:

- **Ciclos 4 e 5:** A instrução I2 fica retida no estágio *ID* esperando I1 gravar t_0 no *WB*.
- **Ciclos 7 e 8:** A instrução I3 fica retida no estágio *ID* esperando I2 gravar t_1 no *WB*.
- **Ciclos 10 e 11:** A instrução I4 fica retida no estágio *ID* esperando I3 gravar t_2 no *WB*.

Q2 — Pipeline Com *Forwarding*

Não ocorre nenhum *stall*. Diferente do problema de Load-Use analisado na Sequência S3, todas as dependências de S1 são do tipo ALU-ALU (cálculo aritmético seguido de outro cálculo aritmético). O resultado de uma instrução *add* é totalmente calculado e disponibilizado ao final do seu próprio estágio *EX*. Como a instrução seguinte também só precisa desse dado no início do seu respectivo estágio *EX*, o hardware de *forwarding* consegue repassar o valor diretamente do registrador de pipeline (*EX/MEM*) para a entrada da ALU da instrução seguinte no exato ciclo de clock necessário, resolvendo o hazard perfeitamente sem congelar o fluxo.

Q3 — Escalonamento Estático

Não é possível eliminar os *stalls* via reordenamento estático neste código. Para que o compilador (ou programador) elimine *stalls* reordenando instruções, é necessário haver instruções independentes que possam ser intercaladas entre as dependentes.

Na Sequência S1, há uma cadeia de dependência estrita e sequencial: I2 depende de I1; I3 depende de I2; I4 depende de I3. Não existe nenhuma instrução neutra no bloco que possa ser intercalada. Qualquer alteração na ordem (por exemplo, colocar I4 antes de I3) quebraria a semântica do programa e alteraria o resultado final da soma acumulada.

Q4 — Impacto da Inserção de Instruções Independentes

Se inserirmos 1 instrução independente entre cada par de instruções dependentes, a distância entre produtora e consumidora passa a ser de 2 ciclos. Assumindo a arquitetura sem *forwarding*:

- A instrução geradora estará no estágio *MEM* quando a consumidora entrar no estágio *ID*. Como o dado só é gravado no banco no estágio *WB* (ciclo seguinte), a instrução consumidora ainda precisará sofrer **1 ciclo de stall** por dependência.

O cálculo de desempenho é apresentado abaixo:

$$\text{Ciclos totais} = N_{\text{instr}} + N_{\text{est}} + N_{\text{stalls}} = 15 + 4 + 6 = 25 \text{ ciclos} \quad (1)$$

$$\text{CPI} = \frac{\text{Ciclos totais}}{N_{\text{instr}}} = \frac{25}{15} \approx 1,67 \quad (2)$$

onde $N_{instr} = 15$ (9 originais + 6 independentes inseridas), $N_{est} = 4$ (ciclos de preenchimento do pipeline) e $N_{stalls} = 6$ (1 *stall* residual por dependência, com distância 2 entre produtora e consumidora).

4.2 Análise da Sequência S2 — Loop com Dependência de Variável de Indução

Resultado das Simulações

A Tabela 6 apresenta os resultados das simulações da Sequência S2 sob diferentes configurações de pipeline escalar.

Tabela 6. Resultados das simulações — Sequência S2

Configuração	Instruções	Ciclos Totais	Stalls / Flushes	CPI
Sem <i>forwarding</i> , sem predição	51	71	– / –	1,39
Com <i>forwarding</i> , sem predição	46	74	8 / 7	1,61
Com <i>forwarding</i> + predição estática	46	74	8 / 7	1,61
Com <i>forwarding</i> + predição dinâmica 1-bit	46	64	8 / 2	1,39
Com <i>forwarding</i> + predição dinâmica 2-bits	46	66	8 / 3	1,43

Q1 — Hazards de Dados e Controle

O loop da Sequência S2 apresenta um hazard de dados evidente entre `lw x13` e `add` (Load-Use), além de um hazard de controle no `bne`.

(a) **Sem *Forwarding*.** Sem encaminhamento, a instrução dependente deve aguardar a gravação do dado no estágio *WB*. O `add` precisa de `x13` (produzido no *WB* do `lw`) e o `bne` precisa de `x11` (produzido no *WB* do `addi`).

Tabela 7. Diagrama de espaço-tempo — S2 sem *forwarding*

Instrução (It. 1)	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14
<code>lw x13, 0(x12)</code>	IF	ID	EX	MEM	WB									
<code>add x10, x10, x13</code>		IF	ID	–	–	EX	MEM	WB						
<code>addi x12, x12, 4</code>			IF	–	–	ID	EX	MEM	WB					
<code>addi x11, x11, -1</code>						IF	ID	EX	MEM	WB				
<code>bne x11, zero, loop</code>							IF	ID	–	–	EX	MEM	WB	
(flush)								IF	–	–	ID	(f)		
(It. 2) <code>lw x13</code>												IF	ID	EX

O total de **stalls de dados por iteração é de 4 bolhas**: 2 geradas pelo Load-Use em `x13` e 2 geradas pela dependência do branch em `x11`.

(b) **Com *Forwarding*.** O repasse de dados resolve o problema do branch, encaminhando `x11` do final do estágio *EX* para a avaliação. Porém, o hazard de Load-Use é irreduzível: o dado lido só existe ao final do estágio *MEM*, tornando impossível o encaminhamento para o *EX* da instrução seguinte no mesmo ciclo.

Tabela 8. Diagrama de espaço-tempo — S2 com *forwarding*

Instrução (It. 1)	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11
lw x13, 0(x12)	IF	ID	EX	MEM	WB						
add x10, x10, x13		IF	ID	–	EX	MEM	WB				
addi x12, x12, 4			IF	–	ID	EX	MEM	WB			
addi x11, x11, -1					IF	ID	EX	MEM	WB		
bne x11, zero, loop						IF	ID	EX	MEM	WB	
(flush)							IF	ID	(f)		
(It. 2) lw x13								IF	ID	EX	MEM

O total de **stalls de dados por iteração cai para apenas 1 bolha** (irredutível do Load-Use).

Q2 — Escalonamento Estático

Para eliminar a única bolha restante com *forwarding* habilitado, devemos separar a instrução geradora (lw) da consumidora (add), preenchendo o intervalo com uma instrução independente.

```

1 loop:
2   lw    x13, 0(x12)      # Le da memoria
3   addi  x12, x12, 4      # (Movida) Preenche a bolha do load-use
4   !
5   add   x10, x10, x13    # x13 ja disponivel via forwarding
6   addi  x11, x11, -1
7   bne   x11, zero, loop

```

Listing 3. Código reordenado — S2 com escalonamento estático (0 stalls de dados)

Justificativa e preservação semântica: A instrução `addi x12, x12, 4` altera apenas o ponteiro do array. O `lw` já efetuou o cálculo do endereço (usando o valor antigo de `x12`) em seu estágio *EX*, antes que o `addi` possa modificar o registrador. O `add` depende apenas de `x10` e `x13`, não sofrendo impacto com o avanço antecipado do ponteiro. Resultado: **0 stalls de dados**.

Q3 — Comparação de Preditores de Desvio

Analisando o padrão de saltos real de 8 iterações — TTTTNTTN (7 Tomados, 1 Não Tomado) — com ambos os preditores iniciando no estado frio “Não Tomado”:

Preditor de 1 Bit:

- Iteração 1 (T): Erra (muda para T)
- Iterações 2 a 7 (T): Acerta todas
- Iteração 8 (N): Erra (muda para N)
- **Resultado: 2 erros**

Preditor de 2 Bits:

- Iteração 1 (T): Erra (SNT → WNT)
- Iteração 2 (T): Erra (WNT → WT)
- Iterações 3 a 7 (T): Acerta todas

- Iteração 8 (N): Erra (ST → WT)
- **Resultado: 3 erros**

Conclusão: Nesta execução inicial e isolada, o preditor de 1 bit erra menos (2 vs. 3 erros). O preditor de 2 bits possui maior inércia e leva dois ciclos para “aquecer”. No entanto, caso o código entrasse no loop novamente em seguida, o preditor de 2 bits preservaria a predição Tomado (estado *Weakly Taken*), tornando-se superior a longo prazo.

Q4 — Branch Delay Slot

O *Branch Delay Slot* é um recurso arquitetural clássico de processadores MIPS onde a instrução imediatamente após um desvio (`bne`) é **sempre executada**, independentemente do salto ser tomado ou não. Isso expõe a latência do branch ao compilador.

Na Sequência S2, o compilador poderia preencher o *slot* de atraso com o decremento do contador (`addi x11, x11, -1`), que é independente do desvio:

```

1 loop:
2     lw    x13, 0(x12)
3     addi  x12, x12, 4
4     add   x10, x10, x13
5     bne   x11, zero, loop    # Desvio
6     addi  x11, x11, -1       # Delay slot: executado sempre

```

Listing 4. Branch Delay Slot aplicado à S2

Se bem aproveitado, o *Delay Slot* elimina completamente a penalidade de controle, substituindo uma bolha forçada pelo hardware por execução de código útil e melhorando o CPI. Caso não haja instrução independente viável, o compilador é obrigado a inserir um NOP, restaurando a penalidade de 1 ciclo.

5. Análise da Sequência S3

Resultado das Simulações

A Tabela 9 apresenta os resultados das simulações da Sequência S3 sob diferentes configurações de pipeline escalar.

Tabela 9. Resultados das simulações — Sequência S3

Configuração	Instruções	Ciclos Totais	Stalls / Flushes	CPI
Sem <i>forwarding</i> , sem predição	11	15	– / –	1,36
Com <i>forwarding</i> , sem predição	11	20	5 / 0	1,82
Com <i>forwarding</i> + predição estática	11	20	5 / 0	1,82
Com <i>forwarding</i> + predição dinâmica 1-bit	11	20	5 / 0	1,82
Com <i>forwarding</i> + predição dinâmica 2-bits	11	20	5 / 0	1,82

Q1 — Hazard de Load-Use

O diagrama abaixo foca nas duas primeiras instruções do bloco principal para ilustrar o hazard de Load-Use, assumindo que a arquitetura possui *forwarding* habilitado.

Tabela 10. Diagrama de espaço-tempo — Load-Use com *forwarding* (S3)

Instrução	C1	C2	C3	C4	C5	C6	C7
lw x10, 0(x5)	IF	ID	EX	MEM	WB		
add x11, x10, x6		IF	ID	–	EX	MEM	WB

Justificativa Analítica.

- **Disponibilidade:** O valor lido da memória e gravado no registrador x10 só estará fisicamente disponível no registrador de pipeline MEM/WB ao final do ciclo 4 (estágio *MEM* do lw).
- **Necessidade:** A instrução add precisa desse valor na entrada da ALU no início do ciclo 4 (estágio *EX* do add original, caso não houvesse bolha).
- **Por que o *forwarding* não resolve?** Porque o repasse exigiria uma “viagem no tempo”: o dado precisaria ir do *final* do ciclo 4 para o *início* do próprio ciclo 4. Como isso é fisicamente impossível, o processador é forçado a injetar 1 ciclo de bolha (*stall*) no ciclo 4. Assim, o estágio *EX* do add é empurrado para o ciclo 5, onde o *forwarding* finalmente consegue repassar o dado da saída do lw (estágio *WB*) para a entrada da ALU do add (estágio *EX*).

Q2 — *Forwarding* e Stall de Load-Use

Ao habilitar o *forwarding* (modelo de 5 estágios), verifica-se que o *stall* do Load-Use continua ocorrendo.

- **O que o diagrama do simulador mostra:** O simulador exibe a instrução dependente (add) “congelada” no estágio *ID* por um ciclo adicional, enquanto uma bolha (operação vazia, *NOP*) é propagada para o estágio *EX*. Na interface, isso é frequentemente contabilizado como um *Data Hazard* explícito, provando que o repasse ALU–ALU funciona perfeitamente, mas o repasse Memória–ALU exige, obrigatoriamente, um ciclo de latência em instruções consecutivas.

Q3 — Escalonamento Estático

Para eliminar os 3 *stalls* de Load-Use presentes no código original, o compilador (ou programador) pode agrupar todas as leituras de memória (lw) no início e todas as operações aritméticas na sequência. Isso afasta a instrução geradora da instrução consumidora, dando tempo para o dado chegar.

```

1      la      x5, base
2
3      lw      x10, 0(x5)          # Inicia Leitura 1
4      lw      x12, 4(x5)          # Inicia Leitura 2 (Cobre o stall de
                                   x10)
5      lw      x14, 8(x5)          # Inicia Leitura 3 (Cobre o stall de
                                   x12)
6
7      add     x11, x10, x6         # x10 ja esta pronto! (Zero stalls)
8      sub     x13, x12, x6         # x12 ja esta pronto! (Zero stalls)

```

```

9      add    x15, x14, x11    # x14 e x11 ja estao prontos (via
                                forwarding)
10
11      sw     x15, 12(x5)

```

Listing 5. Código reordenado — S3 com escalonamento estático (0 stalls)

Cálculo do Speedup (baseado nos dados do Ripes).

- **Versão original:** 20 ciclos totais (CPI = 1,82).
- **Versão reordenada:** Ao eliminar os 3 *stalls* de Load-Use, o tempo total cai para 17 ciclos ($20 - 3 = 17$).
- **Novo CPI:** $\frac{17 \text{ ciclos}}{11 \text{ instruções}} \approx 1,55$.
- **Speedup:**

$$\text{Speedup} = \frac{\text{Ciclos}_{\text{original}}}{\text{Ciclos}_{\text{reordenado}}} = \frac{20}{17} \approx 1,18$$

- **Conclusão:** O código reordenado executa cerca de 17% mais rápido, sem exigir nenhuma alteração no hardware.

Q4 — Latência Estendida de Load

Se a latência de um Load é de 4 ciclos (mesmo com acerto na cache L1), a distância segura entre a instrução de leitura e a instrução que consome o dado deve ser de 4 ciclos.

- **Quantos stalls ocorreriam?** Em um código linear sequencial com `lw` imediatamente seguido de `add`, a arquitetura precisaria inserir **3 stalls** (bolhas) consecutivos por cada dependência. No código S3 original — que possui 3 Load-Use diretos — isso geraria 9 ciclos de atraso apenas por espera de memória.
- **Como o processador mitiga isso?** Processadores modernos como o Cortex-A53 mitigam essa latência por meio de três estratégias principais:
 1. **Execução Fora de Ordem (*Out-of-Order Execution* / Tomasulo):** O processador detecta dinamicamente quais instruções estão aguardando dados e as mantém em espera, buscando instruções adiante no código que já estão prontas para executar.
 2. **Pipelines Superescalares (*Dual-Issue*):** Executam múltiplas instruções por ciclo de clock, absorvendo as bolhas mais rapidamente ao paralelizar tarefas independentes.
 3. **Escalonamento Avançado de Compilador (*Loop Unrolling* / *Software Pipelining*):** O compilador agrupa dezenas de Loads antecipadamente, muito antes das somas — semelhante ao realizado na Q3, mas em grande escala.

6. Análise da Sequência S4

6.1 Resultado das Simulações

A Tabela 11 apresenta os resultados das simulações da Sequência S4 sob diferentes configurações de pipeline escalar.

Tabela 11. Resultados das simulações — Sequência S4

Configuração	Instruções	Ciclos Totais	Stalls / Flushes	CPI
Sem <i>forwarding</i> , sem predição	15	19	– / –	1,27
Com <i>forwarding</i> , sem predição	15	21	2 / 0	1,40
Com <i>forwarding</i> + predição estática	15	21	2 / 0	1,40
Com <i>forwarding</i> + predição dinâmica 1-bit	15	21	2 / 0	1,40
Com <i>forwarding</i> + predição dinâmica 2-bits	15	21	2 / 0	1,40

Q1 — Classificação de Dependências

Para classificar as dependências, mapeamos quem lê e quem escreve em cada registrador. O código possui 13 instruções úteis (ignorando o `addi a7` e o `ecall` finais do sistema operacional).

- **RAW** (*True Data Dependency* — Leitura Após Escrita): a instrução dependente precisa do resultado da anterior.
- **WAR** (Anti-dependência — Gravação Após Leitura): uma instrução tenta sobrescrever um registrador que a instrução anterior ainda precisa ler.
- **WAW** (Dependência de Saída — Gravação Após Gravação): duas instruções tentam gravar no mesmo registrador.

Tabela 12. Dependências identificadas na Sequência S4

Par	Instrução produtora	Tipo	Instrução consumidora / Registrador
I1 → I8	<code>addi x2, zero, 10</code>	RAW	<code>add x1, x2, x3</code> lê x2
I2 → I8	<code>addi x3, zero, 5</code>	RAW	<code>add x1, x2, x3</code> lê x3
I3 → I9	<code>addi x5, zero, 3</code>	RAW	<code>sub x4, x1, x5</code> lê x5
I4 → I10	<code>addi x6, zero, 8</code>	RAW	<code>add x2, x6, x7</code> lê x6
I5 → I10	<code>addi x7, zero, 2</code>	RAW	<code>add x2, x6, x7</code> lê x7
I6 → I11	<code>addi x8, zero, 4</code>	RAW	<code>mul x1, x4, x8</code> lê x8
I7 → I13	<code>addi x9, zero, 6</code>	RAW	<code>sub x4, x5, x9</code> lê x9
I8 → I9	<code>add x1, x2, x3</code>	RAW	<code>sub x4, x1, x5</code> lê x1
I9 → I11	<code>sub x4, x1, x5</code>	RAW	<code>mul x1, x4, x8</code> lê x4
I10 → I12	<code>add x2, x6, x7</code>	RAW	<code>add x5, x1, x2</code> lê x2
I11 → I12	<code>mul x1, x4, x8</code>	RAW	<code>add x5, x1, x2</code> lê x1
I12 → I13	<code>add x5, x1, x2</code>	RAW	<code>sub x4, x5, x9</code> lê x5
I1 → I10	<code>addi x2, zero, 10</code>	WAR	<code>add x2, x6, x7</code> sobrescreve x2
I8 → I11	<code>add x1, x2, x3</code>	WAR	<code>mul x1, x4, x8</code> sobrescreve x1
I9 → I11	<code>sub x4, x1, x5</code>	WAW	<code>mul x1, x4, x8</code> também grava em x1 (via I8)

Q2 — Renomeação de Registradores (RAT)

A Tabela de Alias de Registradores (RAT) elimina falsas dependências (WAR e WAW) apontando registradores arquiteturais (ex: x1, x2) para registradores físicos ocultos no hardware (ex: P1, P2, P3...).

Assumiremos que os registradores inicialmente mapeiam para si mesmos (x1 → P1, x2 → P2...) e alocaremos novos registradores físicos a partir de P10 para cada nova instrução de escrita.

Tabela 13. Renomeação de registradores via RAT — Sequência S4

Instrução Original	Operação de Renomeação	Instrução Renomeada (Despachada)
I1: addi x2, zero, 10	RAT[x2] ← P10	addi P10, zero, 10
I2: addi x3, zero, 5	RAT[x3] ← P11	addi P11, zero, 5
I3: addi x5, zero, 3	RAT[x5] ← P12	addi P12, zero, 3
I4: addi x6, zero, 8	RAT[x6] ← P13	addi P13, zero, 8
I5: addi x7, zero, 2	RAT[x7] ← P14	addi P14, zero, 2
I6: addi x8, zero, 4	RAT[x8] ← P15	addi P15, zero, 4
I7: addi x9, zero, 6	RAT[x9] ← P16	addi P16, zero, 6
I8: add x1, x2, x3	Lê RAT[x2, x3]; RAT[x1] ← P17	add P17, P10, P11
I9: sub x4, x1, x5	Lê RAT[x1, x5]; RAT[x4] ← P18	sub P18, P17, P12
I10: add x2, x6, x7	Lê RAT[x6, x7]; RAT[x2] ← P19	add P19, P13, P14
I11: mul x1, x4, x8	Lê RAT[x4, x8]; RAT[x1] ← P20	mul P20, P18, P15
I12: add x5, x1, x2	Lê RAT[x1, x2]; RAT[x5] ← P21	add P21, P20, P19
I13: sub x4, x5, x9	Lê RAT[x5, x9]; RAT[x4] ← P22	sub P22, P21, P16

Q3 — Caminho Crítico e ILP Máximo

Após a renomeação, o código é representado como um grafo direcionado, fluindo de cima para baixo. Para encontrar o Caminho Crítico, buscamos a cadeia mais longa de instruções com dependência RAW sequencial.

- **Trilha A:** I1/I2 → I8 → I9 → I11 → I12 → I13 (6 nós)
- **Trilha B:** I3 → I9 → I11 → I12 → I13 (5 nós)
- **Trilha C:** I4/I5 → I10 → I12 → I13 (4 nós)

Caminho Crítico: A Trilha A (I1 → I8 → I9 → I11 → I12 → I13) possui o maior comprimento, com **6 instruções**. O tempo total de execução do programa jamais será menor do que o tempo necessário para executar essas 6 instruções em sequência.

ILP Máximo (*Instruction-Level Parallelism*): o ILP máximo ideal é a razão entre o total de instruções úteis e o tamanho do caminho crítico:

$$\text{ILP}_{\max} = \frac{13}{6} \approx 2,16$$

Q4 — Execução *Out-of-Order*

Em um processador *Out-of-Order* capaz de buscar e despachar 2 instruções por ciclo, instruções sem pendências RAW são enviadas diretamente para as Estações de Reserva (*Reservation Stations*) e executadas no mesmo ciclo em que os dados estão prontos, ignorando a ordem original do código.

Tabela 14. Simulação de despacho *Out-of-Order* — Sequência S4

Ciclo	Despacho (Busca)	Execução Efetiva (OoO)	Status das Dependências
1	I1, I2	I1, I2	Constantes. Executam imediatamente.
2	I3, I4	I3, I4	Constantes. Executam imediatamente.
3	I5, I6	I5, I6	Constantes. Executam imediatamente.
4	I7, I8	I7, I8	I7 é constante. I8 dependia de I1 e I2 (já resolvidos no ciclo 1). Ambas executam.
5	I9, I10	I10, I9	I10 depende de I4/I5 (resolvidos). I9 depende de I3 (resolvido) e de I8 (resolvido no ciclo 4). Ambas executam.
6	I11, I12	I11	I11 depende de I6/I9 (resolvidos), logo executa. I12 fica parada aguardando I11 terminar.
7	I13	I12	I13 é despachada, mas fica parada esperando I12. I12 finalmente executa pois I11 terminou no ciclo 6.
8	(Fim Busca)	I13	I13 executa, pois I12 terminou no ciclo 7.

Conclusão do Despacho: Esta simulação prova a eficiência do *Out-of-Order*. No ciclo 5, a instrução I10 foi executada em paralelo e de forma totalmente independente de I9. Todo o programa útil foi condensado e resolvido em apenas **8 ciclos de execução efetiva**, aproximando-se do limite teórico imposto pelo caminho crítico.

7. Análise da Sequência S5

7.1 Resultado das Simulações

A Tabela 15 apresenta os resultados das simulações da Sequência S5 sob diferentes configurações de pipeline escalar.

Tabela 15. Resultados das simulações — Sequência S5

Configuração	Instruções	Ciclos Totais	Stalls / Flushes	CPI
Sem <i>forwarding</i> , sem predição	—	—	— / —	—
Com <i>forwarding</i> , sem predição	105	169	10 / 24	1,61
Com <i>forwarding</i> + predição estática	105	169	10 / 24	1,61
Com <i>forwarding</i> + predição dinâmica 1-bit	105	181	10 / 30	1,72
Com <i>forwarding</i> + predição dinâmica 2-bits	105	169	10 / 24	1,61

7.2 Q1 — Comparação de Preditores de Desvio

Padrão real de ocorrências (10 iterações): T, N, N, N, T, N, N, T, N, N. Premissa: ambos os preditores iniciam no estado “frio” (Não Tomado / *Strongly Not Taken*).

Preditor de 1 Bit.

1. T → Previsão: N | **Erra** (muda para T)
2. N → Previsão: T | **Erra** (muda para N)
3. N → Previsão: N | Acerta (mantém N)
4. N → Previsão: N | Acerta (mantém N)

5. T → Previsão: N | **Erra** (muda para T)
6. N → Previsão: T | **Erra** (muda para N)
7. N → Previsão: N | Acerta (mantém N)
8. T → Previsão: N | **Erra** (muda para T)
9. N → Previsão: T | **Erra** (muda para N)
10. N → Previsão: N | Acerta (mantém N)

Resultado 1-bit: 4 acertos / 6 erros. Taxa de acerto: 40%.

Preditor de 2 Bits.

1. T → SNT (N) | **Erra** (muda para WNT)
2. N → WNT (N) | Acerta (volta para SNT)
3. N → SNT (N) | Acerta (mantém SNT)
4. N → SNT (N) | Acerta (mantém SNT)
5. T → SNT (N) | **Erra** (muda para WNT)
6. N → WNT (N) | Acerta (volta para SNT)
7. N → SNT (N) | Acerta (mantém SNT)
8. T → SNT (N) | **Erra** (muda para WNT)
9. N → WNT (N) | Acerta (volta para SNT)
10. N → SNT (N) | Acerta (mantém SNT)

Resultado 2-bits: 7 acertos / 3 erros. Taxa de acerto: 70%.

7.3 Q2 — Comparação de Preditores de Desvio (Padrão Alternado)

Padrão real de ocorrências (9 avaliações): N, T, N, N, T, N, N, T, N.

Preditor de 1 Bit.

1. N → Previsão: N | Acerta (mantém N)
2. T → Previsão: N | **Erra** (muda para T)
3. N → Previsão: T | **Erra** (muda para N)
4. N → Previsão: N | Acerta (mantém N)
5. T → Previsão: N | **Erra** (muda para T)
6. N → Previsão: T | **Erra** (muda para N)
7. N → Previsão: N | Acerta (mantém N)
8. T → Previsão: N | **Erra** (muda para T)
9. N → Previsão: T | **Erra** (muda para N)

Resultado 1-bit: 3 acertos / 6 erros. Taxa de acerto: 33,3%.

Preditor de 2 Bits.

1. N → SNT (N) | Acerta (mantém SNT)
2. T → SNT (N) | **Erra** (muda para WNT)
3. N → WNT (N) | Acerta (volta para SNT)
4. N → SNT (N) | Acerta (mantém SNT)
5. T → SNT (N) | **Erra** (muda para WNT)
6. N → WNT (N) | Acerta (volta para SNT)
7. N → SNT (N) | Acerta (mantém SNT)
8. T → SNT (N) | **Erra** (muda para WNT)
9. N → WNT (N) | Acerta (volta para SNT)

Resultado 2-bits: 6 acertos / 3 erros. Taxa de acerto: 66,7%.

7.4 Q3 — Cálculo Analítico de CPI

Parâmetros base: 105 instruções totais; 10 bolhas de Load-Use inescapáveis; ciclos base = $105 + 4 \text{ (fill)} + 10 \text{ (load-use)} = 119$ ciclos. Além dos desvios `blt` e `beq`, o loop possui um `j` incondicional (tomado 10 vezes) e um `bge` (tomado apenas na saída, 1 vez). Penalidade fixada: 1 ciclo por flush.

(a) Sem predição (*Always Not-Taken*).

- Erros (sempre que for Tomado): $3(\text{blt}) + 3(\text{beq}) + 10(\text{j}) + 1(\text{bge}) = 17$ erros.
- Ciclos totais: $119 + 17 = 136$ ciclos.
- $\text{CPI} = 136/105 = 1,29$

(b) Preditor de 1 Bit.

- Erros mapeados: $6(\text{blt}) + 6(\text{beq}) + 1(1^\circ \text{ j}) + 1(\text{bge}) = 14$ erros.
- Ciclos totais: $119 + 14 = 133$ ciclos.
- $\text{CPI} = 133/105 = 1,26$

(c) Preditor de 2 Bits.

- Erros mapeados: $3(\text{blt}) + 3(\text{beq}) + 2(\text{aquecimento do j}) + 1(\text{bge}) = 9$ erros.
- Ciclos totais: $119 + 9 = 128$ ciclos.
- $\text{CPI} = 128/105 = 1,21$

7.5 Q4 — Impacto de Dados Ordenados na Predição

Se o array fosse ordenado (ex: $[-4, -3, -1, 0, 0, 0, 2, 3, 5, 7]$), o comportamento caótico dos desvios desapareceria.

- O desvio B1 (`blt`) passaria a ter padrão agrupado: T, T, T, N, N, N, N, N, N, N.
- O desvio B2 (`beq`) passaria a ter padrão: T, T, T, N, N, N, N.
- **Quem mais se beneficia:** ambos os preditores dinâmicos (1-bit e 2-bits) se beneficiariam enormemente, pois aprenderiam o padrão após o primeiro ou segundo salto e acertariam todas as verificações seguintes em sequência. O preditor estático não veria tanto benefício, pois continuaria errando os valores na transição entre grupos. A ordenação prévia de dados é uma técnica clássica para otimizar preditores de desvio em algoritmos de alta performance.

7.6 Q5 — *Branchless Programming*

Em vez de depender de hardware preditivo para “adivinhar” dados caóticos, a melhor transformação é o ***Branchless Programming*** (programação sem desvios). Os desvios condicionais são substituídos por instruções que geram valores booleanos (0 ou 1) nativamente na ALU, como a instrução `slt` (*Set Less Than*).

Exemplo Prático — Eliminando o desvio B1. Em vez de um `blt` para decidir se incrementa o contador de negativos, utiliza-se:

```
1 slt x14, x11, zero      # Se x11 < 0, x14 recebe 1. Senao, recebe
    0.
2 add x7, x7, x14         # Soma x14 diretamente ao contador de
    negativos.
```

Listing 6. Branchless Programming — substituição do desvio B1

Isso elimina o hazard de controle (*Control Hazard*) e o transforma em uma dependência de dados simples, que flui perfeitamente no pipeline sem nenhum risco de erro de predição ou necessidade de flush.

8. Pipeline Escalar e Superescalar

O pipeline é uma técnica utilizada em arquiteturas de processadores para aumentar o paralelismo e melhorar o desempenho da execução de instruções. Em vez de executar uma instrução completamente antes de iniciar a próxima, o pipeline divide a execução em estágios, permitindo que múltiplas instruções sejam processadas simultaneamente em diferentes fases.

No modelo escalar tradicional, o processador emite apenas uma instrução por ciclo de clock. Já no modelo superescalar, múltiplas instruções podem ser despachadas simultaneamente, desde que não existam dependências de dados ou conflitos estruturais. Dessa forma, arquiteturas superescalares exploram paralelismo em nível de instrução (*Instruction-Level Parallelism – ILP*) para aumentar o throughput do processador.

Entre as técnicas modernas utilizadas em arquiteturas superescalares destacam-se:

- Execução fora de ordem (*Out-of-Order Execution*);
- Renomeação de registradores;
- Predição de desvios;
- Uso de Reservation Stations;
- Reorder Buffer (ROB).

Esses mecanismos permitem reduzir o impacto de dependências entre instruções e melhorar a utilização das unidades funcionais do processador.

9. Algoritmo de Tomasulo

O algoritmo de Tomasulo é um método de escalonamento dinâmico criado para permitir execução fora de ordem em processadores superescalares. Seu principal objetivo é minimizar atrasos causados por dependências de dados e aumentar o paralelismo durante a execução das instruções.

O algoritmo utiliza estruturas como:

- **Reservation Stations (RS):** armazenam instruções aguardando operandos;
- **Common Data Bus (CDB):** barramento utilizado para disseminar resultados produzidos pelas unidades funcionais;
- **Reorder Buffer (ROB):** garante que o commit das instruções ocorra em ordem correta.

Além disso, o algoritmo elimina dependências falsas através da técnica de renomeação de registradores. Dessa forma, diferentes instruções podem utilizar versões distintas de um mesmo registrador lógico sem gerar conflitos artificiais.

10. Q1. Análise da Execução

A sequência de instruções utilizada na simulação foi:

```
ld f6, 34(r2)
ld f2, 45(r3)
multd f0, f2, f4
subd f8, f6, f2
divd f10, f0, f6
addd f6, f8, f2
```

A Tabela 18 apresenta os ciclos de issue, execução, escrita de resultado e commit de cada instrução durante a simulação.

Tabela 16. Instruction Status

Instrução	Issue	Exec. Início	Exec. Fim	Write Result	Commit
ld f6, 34(r2)	0	2	2	3	4
ld f2, 45(r3)	1	3	3	4	5
multd f0, f2, f4	2	7	7	8	9
subd f8, f6, f2	3	5	5	6	10
divd f10, f0, f6	4	15	15	16	17
addd f6, f8, f2	5	7	7	8	18

Observa-se que as instruções de carga (ld) foram executadas primeiro, permitindo posteriormente a execução das instruções dependentes. A instrução multd precisou aguardar a disponibilidade do registrador f2, enquanto a instrução divd permaneceu bloqueada até que o resultado de multd fosse disponibilizado no barramento comum de dados.

11. Q2. Reservation Stations em Espera

Durante a execução, algumas Reservation Stations permaneceram ocupadas aguardando operandos provenientes de outras instruções. A Tabela 17 mostra os principais momentos de espera identificados.

Tabela 17. Reservation Stations em espera

Ciclo	RS	Dependência
2	Mult1	Esperando resultado de f2 ($Q_j = 1$)
3	Add1	Esperando resultado de f2 ($Q_k = 1$)
4	Mult2	Esperando resultado de f0 ($Q_j = 2$)
5	Add2	Esperando resultado de f8 ($Q_j = 3$)

Essas dependências demonstram o funcionamento do mecanismo de rastreamento de operandos do algoritmo de Tomasulo. Enquanto os operandos não estavam disponíveis, as instruções permaneciam armazenadas nas Reservation Stations sem bloquear completamente o restante do pipeline.

12. Q4. Execução da instrução DIVD

A instrução:

```
divd f10, f0, f6
```

foi emitida no ciclo 4, porém permaneceu aguardando o resultado da instrução:

```
multd f0, f2, f4
```

O início da execução da instrução DIVD ocorreu apenas no ciclo 15, após a disponibilidade dos operandos necessários.

- Início da execução: ciclo 15
- Término da execução / Write Result: ciclo 16
- Commit: ciclo 17

Esse comportamento evidencia a latência elevada típica da operação de divisão em ponto flutuante, além da dependência direta em relação ao resultado produzido pela multiplicação.

13. Q5. Conflito no Common Data Bus (CDB)

Sim, ocorreu conflito no barramento comum de dados (CDB).

No ciclo 8, duas instruções finalizaram praticamente ao mesmo tempo:

- `multd f0, f2, f4`
- `addd f6, f8, f2`

Como o barramento CDB permite apenas uma escrita por ciclo, uma das instruções precisou utilizar o barramento antes da outra.

Observando a simulação, nota-se que a instrução `multd` teve prioridade de escrita no CDB, produzindo o valor V2. Já a instrução `addd` teve seu resultado disponibilizado posteriormente como V5.

Esse tipo de conflito é comum em arquiteturas superescalares e demonstra uma limitação estrutural importante do barramento de resultados compartilhado.

14. Q1. Tabela *Instruction Status* e comparação com T1

14.1 Tabela *Instruction Status* — T2

A tabela a seguir foi extraída diretamente da saída do simulador ciclo a ciclo.

Tabela 18. Instruction Status — T2 (ciclos obtidos do simulador)

#	Instrução	Issue	Exec. início	Write Result	Commit
I1	DIVD F0, F2, F4	0	8	9	10
I2	MULTD F6, F0, F8	1	12	13	14
I3	ADDD F2, F6, F4	2	14	15	16
I4	MULTD F8, F2, F6	3	18	19	20
I5	SUBD F0, F8, F6	10	20	21	22
I6	ADDD F4, F0, F2	11	22	23	24

Observação sobre I1: embora I1 seja emitida no ciclo 0, ela permanece no estado *Issue* até o ciclo 7 e só inicia a execução no ciclo 8. Isso ocorre porque o *DIVD* exige 8 ciclos de execução; a *RS* aguarda a unidade funcional ficar disponível e os operandos já estão prontos desde o *issue* ($V_j = R[f2]$, $V_k = R[f4]$).

14.2 Comparação entre T1 e T2

Tabela 19. Comparação direta T1 vs. T2 — ciclos extraídos do simulador

Métrica	T1	T2
Write Result final	Ciclo 16 (I5)	Ciclo 23 (I6)
Commit final	Ciclo 18 (I6)	Ciclo 24 (I6)
Instrução mais lenta	DIVD, 8 ciclos (I5)	DIVD, 8 ciclos (I1)
Posição do gargalo	Final da cadeia	Início da cadeia
ILP explorado	Maior	Menor

Em T1, as duas primeiras instruções são loads com apenas 2 ciclos de execução. I1 faz *Write Result* no ciclo 3 e I2 no ciclo 4, destravando rapidamente as dependentes: I3 começa a executar no ciclo 4, I4 no ciclo 5 e I6 no ciclo 7 — todas em sobreposição. O *DIVD* aparece apenas como I5, que começa no ciclo 8 depois que seus dois operandos (F0 de I3 e F6 de I1) já estão prontos; como é a última instrução significativa da cadeia, sua latência de 8 ciclos não bloqueia nada antes dela.

Em T2, o *DIVD* é I1 — a instrução raiz. Toda instrução que depende de F0 (e, por cascata, de F6, F2, F8) fica bloqueada até o ciclo 9. A cadeia de bloqueios é:

- I2 só inicia execução no ciclo 12 (aguarda V0 de I1, que chega via CDB no ciclo 9).
- I3 depende de I2 (RAW em F6) e só executa no ciclo 14.
- I4 depende de I3 e I2 e só executa no ciclo 18.
- I5 e I6 aguardam I4 e I3, respectivamente, executando nos ciclos 20 e 22.

O commit final ocorre no ciclo **24** em T2, contra o ciclo **18** em T1 — uma diferença de **6 ciclos** causada exclusivamente pela posição do DIVD no grafo de dependências.

15. Q2. Valores na RS no Issue de I1 e I2 — eliminação do WAR I1 → I3

15.1 Reservation Station no ciclo de Issue de I1 (ciclo 0)

I1 (DIVD F0, F2, F4) é emitida para **Mult1**. No momento do issue, nenhuma instrução anterior escreve F2 ou F4, logo os valores estão disponíveis diretamente no banco de registradores:

Tabela 20. Estado de Mult1 após o Issue de I1 (ciclo 0)

Vj	Vk	Qj	Qk	Dest (ROB)
R[f2] = 2.0	R[f4] = 3.0	—	—	0

O simulador confirma: Mult1: Vj=R[f2], Vk=R[f4] no Cycle 0.

15.2 Reservation Station no ciclo de Issue de I2 (ciclo 1)

I2 (MULTD F6, F0, F8) é emitida para **Mult2**. F0 ainda não está pronto (I1 está em execução), portanto Qj registra a dependência. F8 está disponível e é capturado imediatamente:

Tabela 21. Estado de Mult2 após o Issue de I2 (ciclo 1)

Vj	Vk	Qj	Qk	Dest (ROB)
—	R[f8] = 0.5	0 (ROB de I1)	—	1

O simulador confirma: Mult2: Vk=R[f8], Qj=0 no Cycle 1.

15.3 Demonstração da eliminação do WAR I1 → I3 (registrador F2)

A anti-dependência WAR ocorre porque I1 *lê* F2 como operando e I3 *escreve* F2 como destino. Em um pipeline out-of-order sem renomeação, se I3 fosse adiantada e escrevesse F2 antes de I1 usar seu valor, I1 leria o valor novo e incorreto. O Tomasulo elimina esse hazard da seguinte forma:

1. No **ciclo 0** (issue de I1), o valor $F2 = 2.0$ é copiado da ARF para **Vj** da RS de I1. I1 passa a ter sua própria cópia privada do valor.
2. I3 só é emitida no **ciclo 2** e só escreve F2 no **ciclo 15** (Write Result).
3. A partir do ciclo 0, I1 não referencia mais o nome arquitetural F2 — usa apenas Vj. Qualquer escrita posterior em F2 é invisível para I1.

Portanto, a captura antecipada do valor no campo Vj garante que a WAR I1 → I3 seja **completamente eliminada** pelo Tomasulo, sem nenhum stall adicional.

16. Q2. Análise do WAW entre I1 e I5 (registrador F0)

Ambas as instruções escrevem F0: I1 produz a versão 1 de F0 e I5 produz a versão final (correta).

16.1 (a) Ciclo em que I1 faz Write Result de F0

I1 faz **Write Result no ciclo 9**. O simulador mostra, no Cycle 9, a entrada 0 do ROB com estado `Write Result`, destino `f0` e valor `V0`.

16.2 (b) Ciclo em que I5 faz Write Result de F0

I5 faz **Write Result no ciclo 21**. O simulador mostra, no Cycle 21, a entrada 4 do ROB com estado `Write Result`, destino `f0` e valor `V4`.

16.3 (c) Register Result Status de F0 antes e depois do Write Result de I1

Tabela 22. Evolução do Register Result Status de F0

Momento	ROB Entry apontado	Significado
Ciclos 0–9 (antes do WR de I1)	Entry 0 (I1)	I1 é a única escritora pendente de F0
Ciclo 10 em diante	Entry 4 (I5)	I5 é a última escritora de F0; seu valor será o definitivo

O simulador confirma: após o commit de I1 no ciclo 10, o Register Status mostra `f0 → entry 4, Busy=Yes`. Quando I5 completa e o ROB faz o commit no ciclo 22, o valor `V4` é gravado na ARF como o valor final e correto de F0.

16.4 (d) Qual valor I2 recebe para F0 — de I1 ou de I5?

I2 recebe o valor **de I1** (V0), que é o correto. O mecanismo pelo qual o Tomasulo garante isso é o seguinte:

1. No issue de I2 (ciclo 1), I2 registra **Qj = 0** (ROB entry de I1), indicando que aguarda o broadcast de I1 pelo CDB.
2. No ciclo 9, I1 transmite V0 pelo CDB com *tag* = 0. A RS de I2 (Mult2) compara seu Qj com o tag transmitido: **0 = 0** → captura V0 em Vj e limpa Qj.
3. I2 inicia a execução no ciclo 12 com Vj = V0.
4. I5 só transmite V4 pelo CDB no ciclo 21, quando I2 já terminou no ciclo 13. Nenhuma RS aguarda o tag de I5 para F0 nesse momento.

A WAW é eliminada porque o ROB controla a ordem de commit: mesmo que I1 escreva F0 antes de I5, o valor arquitetural visível externamente só é atualizado pelo commit, que ocorre em ordem. O valor de I1 (V0) é legítimo para as RSs que o aguardavam; I5 simplesmente sobrescreve F0 na ARF em momento posterior, tornando V4 o valor final.

17. Q4. Renomeação explícita com RAT (Parte B.2)

17.1 Configuração inicial do RAT

Registradores físicos P0–P15. Mapeamento inicial:

F0	F2	F4	F6	F8	demais
P0	P2	P4	P6	P8	identidade

A cada instrução que **escreve** um registrador arquitetural, aloca-se um novo registrador físico livre (P9 em diante). Os operandos fonte são lidos do RAT vigente *no momento do issue*.

17.2 Sequência renomeada e estado do RAT após cada instrução

Tabela 23. Renomeação explícita — instrução renomeada e RAT resultante

#	Original	Renomeada	RAT após (alterações)
I1	DIVD F0, F2, F4	DIVD P9, P2, P4	F0 ← P9
I2	MULTD F6, F0, F8	MULTD P10, P9, P8	F6 ← P10
I3	ADDD F2, F6, F4	ADDD P11, P10, P4	F2 ← P11
I4	MULTD F8, F2, F6	MULTD P12, P11, P10	F8 ← P12
I5	SUBD F0, F8, F6	SUBD P13, P12, P10	F0 ← P13
I6	ADDD F4, F0, F2	ADDD P14, P13, P11	F4 ← P14

17.3 RAT final após todas as instruções

F0	F2	F4	F6	F8	demais
P13	P11	P14	P10	P12	identidade

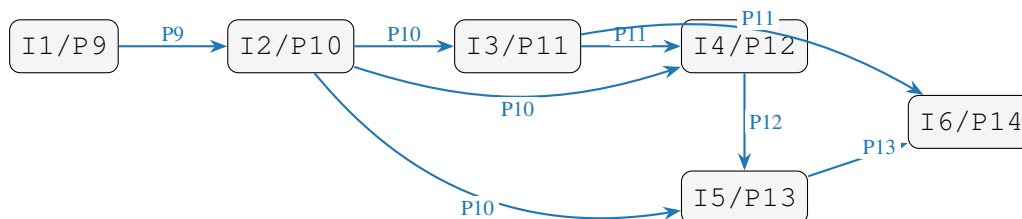
17.4 Dependências eliminadas

Tabela 24. Falsas dependências presentes na sequência original e sua eliminação pela renomeação

Tipo	Par	Reg.	Por que foi eliminada
WAR	I1→I3	F2	I1 lê P2 ; I3 escreve P11 — registradores distintos
WAR	I2→I4	F8	I2 lê P8 ; I4 escreve P12 — registradores distintos
WAR	I2→I5	F0	I2 lê P9 (F0 de I1); I5 escreve P13 — distintos
WAR	I1→I6	F4	I1 lê P4 ; I6 escreve P14 — registradores distintos
WAW	I1→I5	F0	I1 escreve P9 ; I5 escreve P13 — nomes distintos, sem conflito
Total			4 WAR + 1 WAW = 5 falsas dependências eliminadas

17.5 Dependências RAW remanescentes e ILP máximo

Após a renomeação, apenas as dependências verdadeiras (RAW) permanecem:

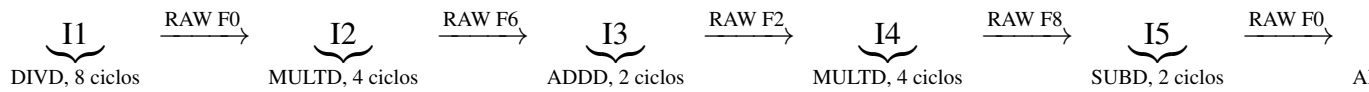


Após a renomeação, I4 e I5 possuem dependências independentes entre si no nível de operandos (ambas aguardam P10 e P12/P13 por caminhos distintos), e podem ser escalonadas em paralelo quando os recursos permitirem. O **ILP máximo** é limitado pelo caminho crítico das RAW remanescentes — detalhado na Questão 5.

18. Q5. Caminho crítico de T2 e comparação com T1

18.1 Caminho crítico de T2

O caminho crítico é a maior cadeia de dependências RAW em termos de ciclos de execução:



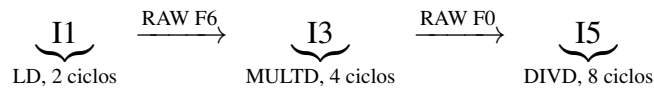
Soma dos ciclos de execução na cadeia crítica:

$$8 + 4 + 2 + 4 + 2 + 2 = \mathbf{22 \text{ ciclos de execução}}$$

Adicionando os overheads de issue e write result, o simulador confirma: **Write Result final no ciclo 23 e Commit final no ciclo 24.**

18.2 Caminho crítico de T1

Em T1, o caminho crítico passa pelas instruções que formam a maior cadeia de dependências RAW:



Soma dos ciclos de execução:

$$2 + 4 + 8 = \mathbf{14 \text{ ciclos de execução}}$$

O simulador confirma: I5 faz Write Result no ciclo 16 e Commit no ciclo 17. A cadeia paralela $I2 \rightarrow I4 \rightarrow I6$ ($2+2+2 = 6$ ciclos de EX) é muito mais curta e não é o gargalo — I6 até faz Write Result no ciclo 8, muito antes de I5. O Commit final de T1 ocorre no ciclo **18** (I6 aguarda o commit em ordem de I5 no ciclo 17).

18.3 Comparação entre os caminhos críticos

Tabela 25. Comparação dos caminhos críticos — T1 vs. T2

Característica	T1	T2
Caminho crítico	$I1 \rightarrow I3 \rightarrow I5$	$I1 \rightarrow I2 \rightarrow I3 \rightarrow I4 \rightarrow I5 \rightarrow I6$
EX na cadeia crítica	$2+4+8 = \mathbf{14 \text{ ciclos}}$	$8+4+2+4+2+2 = \mathbf{22 \text{ ciclos}}$
Write Result final	Ciclo 16	Ciclo 23
Commit final	Ciclo 18	Ciclo 24
Instruções fora do caminho crítico	I2, I4, I6 (executam em paralelo)	Nenhuma (cadeia única)
ILP explorado	Maior	Menor

18.4 Por que T1 permite maior ILP

A diferença fundamental está na **posição do DIVD no grafo de dependências**, não na sua presença. Ambas as sequências têm exatamente um DIVD de 8 ciclos; o que muda é onde ele aparece:

- **T1:** o DIVD é a instrução *final* da cadeia crítica (I5). Enquanto ele executa, as outras instruções já terminaram ou estão em execução em paralelo. Os loads rápidos (2 ciclos cada) liberam os produtores já no ciclo 3–4, permitindo que I3, I4 e I6 executem em sobreposição. O hardware está sendo aproveitado ao máximo durante os 8 ciclos do DIVD.
- **T2:** o DIVD é a instrução *inicial* da cadeia crítica (I1). Nenhuma das cinco instruções seguintes pode avançar até que I1 conclua. As Reservation Stations ficam ocupadas aguardando, mas as unidades funcionais ficam ociosas. A latência de 8 ciclos do DIVD se propaga como multiplicador sobre toda a cadeia.

O Tomasulo elimina as falsas dependências (WAR e WAW) em ambas as sequências, mas **não pode eliminar RAW verdadeiras**. Em T2, toda a cadeia é uma sequência de RAW encadeadas após o DIVD inicial, resultando em 22 ciclos de execução no caminho crítico contra 14 em T1 — uma diferença de **8 ciclos de EX** e **6 ciclos de commit** causada unicamente pelo reordenamento das instruções.

19. Renomeação de Registradores — Análise Manual

A sequência S4 utilizada é composta por seis instruções sobre registradores arquiteturais x1–x9:

#	Instrução	Lê / Escreve
I1	add x1, x2, x3	escreve x1 lê x2, x3
I2	sub x4, x1, x5	escreve x4 lê x1, x5
I3	add x2, x6, x7	escreve x2 lê x6, x7
I4	mul x1, x4, x8	escreve x1 lê x4, x8
I5	add x5, x1, x2	escreve x5 lê x1, x2
I6	sub x4, x5, x9	escreve x4 lê x5, x9

19.1 Identificação de Dependências

19.1.1 Dependências RAW (Read After Write — verdadeiras)

Ocorrem quando uma instrução posterior lê um registrador que uma instrução anterior ainda vai escrever. Não podem ser eliminadas por renomeação.

- I1 → I2 (x1): I2 lê x1 escrito por I1
- I2 → I4 (x4): I4 lê x4 escrito por I2
- I3 → I5 (x2): I5 lê x2 escrito por I3
- I4 → I5 (x1): I5 lê x1 escrito por I4
- I5 → I6 (x5): I6 lê x5 escrito por I5

19.1.2 Dependências WAR (Write After Read — anti-dependências)

Ocorrem quando uma instrução posterior escreve em um registrador que uma instrução anterior ainda vai ler. Em execução fora de ordem, a instrução posterior poderia sobrescrever o valor antes que a anterior o lesse. São **falsas dependências**, eliminadas por renomeação.

- I1 → I3 (x2): I3 escreve x2; I1 lê x2
- I2 → I5 (x5): I5 escreve x5; I2 lê x5
- I4 → I6 (x4): I6 escreve x4; I4 lê x4

19.1.3 Dependências WAW (Write After Write — dependências de saída)

Ocorrem quando duas instruções escrevem no mesmo registrador arquitetural. O valor intermediário pode ser sobrescrito fora de ordem. Também são **falsas dependências**, eliminadas por renomeação.

- I1 → I4 (x1): ambas escrevem x1; o valor final é o de I4
- I2 → I6 (x4): ambas escrevem x4; o valor final é o de I6

19.1.4 Tabela-resumo de dependências

Tabela 26. Matriz de dependências entre instruções da sequência S4

De \ Para	I1	I2	I3	I4	I5	I6
I1	—	RAW	WAR	WAW	—	—
I2	—	—	—	RAW	WAR	WAW
I3	—	—	—	—	RAW	—
I4	—	—	—	—	RAW	WAR
I5	—	—	—	—	—	RAW
I6	—	—	—	—	—	—

19.2 Grafo de Dependências

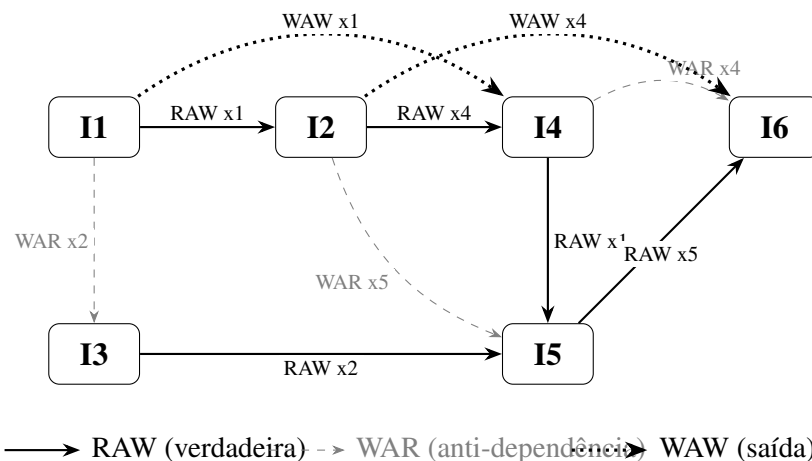


Figura 1. Grafo de dependências da sequência S4

19.3 Aplicação da Renomeação de Registradores (RAT)

Utilizou-se um Register Alias Table (RAT) com 16 registradores físicos (P0–P15) para 9 registradores arquiteturais (x1–x9). Mapeamento inicial:

x1	x2	x3	x4	x5	x6	x7	x8	x9
P1	P2	P3	P4	P5	P6	P7	P8	P9

Registradores físicos livres: P10, P11, P12, P13, P14, P15. A cada instrução que **escreve** um registrador arquitetural, aloca-se o próximo físico livre. Os operandos fonte são lidos do RAT vigente no momento do issue.

19.3.1 Passo 1 — I1: **add x1, x2, x3**

`add P10, P2, P3`

Novo mapeamento: **x1** ← **P10**. Nenhuma dependência afetada neste passo.

19.3.2 Passo 2 — I2: **sub x4, x1, x5**

Operandos lidos do RAT atual: x1 = P10, x5 = P5.

`sub P11, P10, P5`

Novo mapeamento: **x4** ← **P11**.

19.3.3 Passo 3 — I3: **add x2, x6, x7**

Operandos lidos do RAT atual: x6 = P6, x7 = P7.

`add P12, P6, P7`

Novo mapeamento: **x2** ← **P12**.

WAR I1→I3 eliminada: I1 lê **P2**; I3 escreve **P12**. Registradores distintos — sem conflito.

19.3.4 Passo 4 — I4: **mul x1, x4, x8**

Operandos lidos do RAT atual: x4 = P11, x8 = P8.

`mul P13, P11, P8`

Novo mapeamento: **x1** ← **P13**.

WAW I1→I4 eliminada: I1 escreveu **P10**; I4 escreve **P13**. Nomes físicos distintos — sem conflito de saída.

19.3.5 Passo 5 — I5: add x5, x1, x2

Operandos lidos do RAT atual: x1 = P13, x2 = P12.

add P14, P13, P12

Novo mapeamento: **x5** ← **P14**.

WAR I2→I5 eliminada: I2 lê **P5**; I5 escreve **P14**. Registradores distintos — sem conflito.

19.3.6 Passo 6 — I6: sub x4, x5, x9

Operandos lidos do RAT atual: x5 = P14, x9 = P9.

sub P15, P14, P9

Novo mapeamento: **x4** ← **P15**.

WAR I4→I6 eliminada: I4 lê **P11**; I6 escreve **P15**. Registradores distintos — sem conflito.

WAW I2→I6 eliminada: I2 escreveu **P11**; I6 escreve **P15**. Nomes físicos distintos — sem conflito de saída.

19.3.7 Sequência renomeada completa e RAT final

Tabela 27. Sequência renomeada e estado do RAT após cada instrução

#	Original	Renomeada	RAT alterado
I1	add x1, x2, x3	add P10, P2, P3	x1 ← P10
I2	sub x4, x1, x5	sub P11, P10, P5	x4 ← P11
I3	add x2, x6, x7	add P12, P6, P7	x2 ← P12
I4	mul x1, x4, x8	mul P13, P11, P8	x1 ← P13
I5	add x5, x1, x2	add P14, P13, P12	x5 ← P14
I6	sub x4, x5, x9	sub P15, P14, P9	x4 ← P15

Tabela 28. RAT final após todas as instruções

x1	x2	x3	x4	x5	x6	x7	x8	x9
P13	P12	P3	P15	P14	P6	P7	P8	P9

19.3.8 Dependências eliminadas pela renomeação

Tabela 29. Falsas dependências eliminadas pela renomeação

Tipo	Par	Reg.	Por que foi eliminada
WAR	I1→I3	x2	I1 lê P2 ; I3 escreve P12 — nomes distintos
WAR	I2→I5	x5	I2 lê P5 ; I5 escreve P14 — nomes distintos
WAR	I4→I6	x4	I4 lê P11 ; I6 escreve P15 — nomes distintos
WAW	I1→I4	x1	I1 escreve P10 ; I4 escreve P13 — sem conflito
WAW	I2→I6	x4	I2 escreve P11 ; I6 escreve P15 — sem conflito
Total eliminadas			3 WAR + 2 WAW = 5 falsas dependências

19.4 Dependências Remanescentes e ILP Máximo

Após a renomeação, permanecem apenas as dependências RAW verdadeiras, que não podem ser eliminadas por nenhuma técnica de hardware:

- I1 → I2 (P10): I2 lê P10 produzido por I1
- I2 → I4 (P11): I4 lê P11 produzido por I2
- I3 → I5 (P12): I5 lê P12 produzido por I3
- I4 → I5 (P13): I5 lê P13 produzido por I4
- I5 → I6 (P14): I6 lê P14 produzido por I5

O **caminho crítico** é a cadeia RAW mais longa:

$$I1 \xrightarrow{P10} I2 \xrightarrow{P11} I4 \xrightarrow{P13} I5 \xrightarrow{P14} I6$$

Esta cadeia possui **5 instruções sequenciais**, que exigem no mínimo 5 ciclos para serem completadas, mesmo com recursos ilimitados. I3 é a única instrução independente do caminho crítico e pode ser executada em paralelo com I1 ou I2.

O **ILP máximo**, assumindo recursos ilimitados e apenas RAW como restrição, é dado por:

$$ILP_{\max} = \frac{\text{Total de instruções}}{\text{Comprimento do caminho crítico}} = \frac{6}{5} = 1,2 \text{ IPC}$$

19.5 Comparação entre Arquiteturas Superescalares 2-wide

Considera-se um processador superescalar com **2 slots de despacho** (2-wide): em cada ciclo, até 2 instruções podem ser despachadas simultaneamente, desde que não haja conflito de dependências. Assume-se latência de 1 ciclo para todas as operações (modelo simplificado para análise de ILP estrutural).

19.5.1 Superescalar 2-wide Em Ordem (*In-Order*)

No modelo em ordem, as instruções são despachadas respeitando estritamente a sequência do programa. Uma instrução só pode ser despachada se não houver dependência RAW não resolvida com alguma instrução já despachada anteriormente.

Tabela 30. Despacho ciclo a ciclo — Superescalar 2-wide em ordem

Ciclo	Slot 1	Slot 2	Justificativa
1	I1	I3	I1 e I3 são independentes entre si (I3 não depende de I1 por RAW). Em ordem, I3 é a próxima candidata após I1 sem RAW pendente.
2	I2	—	I2 depende de I1 (RAW P10): aguarda 1 ciclo. I3 já foi despachada. Não há outro candidato sem dependência pendente.
3	I4	—	I4 depende de I2 (RAW P11): aguarda I2 completar.
4	I5	—	I5 depende de I4 (RAW P13) e de I3 (RAW P12, já pronto desde ciclo 1).
5	I6	—	I6 depende de I5 (RAW P14).

- **Total de ciclos:** 5
- **Instruções despachadas:** 6
- **IPC alcançado:** $6/5 = 1,2$

O modelo em ordem consegue aproveitar o único paralelismo disponível (I1 e I3 no ciclo 1), atingindo o ILP máximo teórico neste exemplo.

19.5.2 Superescalar 2-wide Fora de Ordem (*Out-of-Order*)

No modelo fora de ordem, a renomeação de registradores eliminou todas as falsas dependências WAR e WAW. O hardware (via Reservation Stations e ROB) pode reordenar livremente as instruções, despachando qualquer instrução cujos operandos físicos estejam disponíveis, independentemente da ordem original do programa.

Tabela 31. Despacho ciclo a ciclo — Superescalar 2-wide fora de ordem

Ciclo	Slot 1	Slot 2	Justificativa
1	I1	I3	I1 e I3 não têm dependências RAW entre si (WAR I1→I3 foi eliminada). OoO pode despachar ambas imediatamente.
2	I2	—	I2 depende de I1 (RAW P10, verdadeira): aguarda. Não há outra instrução pronta — I4 depende de I2; I5 depende de I3 e I4; I6 depende de I5.
3	I4	—	I4 depende de I2 (RAW P11, verdadeira): aguarda I2 completar.
4	I5	—	I5 depende de I4 (RAW P13) e I3 (RAW P12, já pronto). WAR I2→I5 foi eliminada — sem restrição adicional.
5	I6	—	I6 depende de I5 (RAW P14, verdadeira). WAR I4→I6 e WAW I2→I6 foram eliminadas.

- **Total de ciclos:** 5
- **Instruções despachadas:** 6
- **IPC alcançado:** $6/5 = 1,2$

19.5.3 Análise Comparativa

Tabela 32. Comparação entre os modelos superescalares 2-wide

Critério	Em Ordem	Fora de Ordem
Ciclos totais	5	5
IPC alcançado	1,2	1,2
ILP máximo teórico	1,2	1,2
Reordenamento de instruções	Não	Sim
Falsas dep. eliminadas	Não	Sim (RAT)
Complexidade de hardware	Menor	Maior

Neste exemplo específico, ambos os modelos atingem o mesmo IPC de 1,2, pois o **caminho crítico de RAW verdadeiras** (I1→I2→I4→I5→I6) é o único fator limitante. A única instrução paralelizável é I3, que já pode ser adiantada no modelo em ordem junto com I1 no ciclo 1.

A diferença entre os modelos se manifestaria em sequências com mais falsas dependências e mais instruções independentes: nesses casos, o modelo fora de ordem identificaria e executaria instruções prontas que o modelo em ordem seria forçado a manter estagnadas por dependências já eliminadas. Em S4, a densidade de RAW verdadeiras no caminho crítico não deixa espaço para o OoO demonstrar vantagem de IPC sobre o em ordem.

20. Ferramenta de Simulação e Metodologia

Para a realização das simulações superescalares, foi utilizado o **gem5** (versão 23.0.0.1), simulador de arquitetura de computadores de código aberto amplamente adotado em pesquisa acadêmica e industrial. A arquitetura alvo escolhida foi **RISC-V 64 bits**, e o modo de execução utilizado foi o *Syscall Emulation* (SE), que permite simular programas em espaço de usuário sem a necessidade de um sistema operacional completo.

O ambiente de simulação foi configurado sobre o **WSL2** (*Windows Subsystem for Linux 2*), com Ubuntu 22.04, em uma máquina com processador AMD Ryzen 7 5700X3D (8 núcleos, 16 *threads*). O gem5 foi compilado a partir do código-fonte com o comando:

```
scons build/RISCV/gem5.opt -j16
```

Listing 7. Compilação do gem5 para RISC-V

As sequências S1–S5 foram escritas em Assembly RISC-V, compiladas com o *cross-compiler* `riscv64-linux-gnu-gcc` e geradas como binários ELF estáticos:

```
riscv64-linux-gnu-gcc -static -nostdlib -o s${n}.elf s${n}*.s
```

Listing 8. Compilação das sequências

Adaptação do `se.py` para Configurações Superescalares

Uma contribuição relevante deste trabalho foi a necessidade de modificar diretamente o script de configuração padrão do gem5 (`configs/deprecated/example/se.py`) para suportar as cinco configurações arquiteturais exigidas. O script original não expõe parâmetros como largura de despacho, número de entradas do ROB ou configuração das filas de instruções como argumentos de linha de comando para o modelo `RiscvO3CPU`.

A solução adotada foi instrumentar o `se.py` para detectar dinamicamente, pelo nome do diretório de saída passado via `-outdir`, qual configuração está sendo executada (C2, C3, C4 ou C5), e aplicar os parâmetros correspondentes diretamente nos objetos de CPU do gem5. O trecho inserido no `se.py` é apresentado abaixo:

```
1 if args.cpu_type == "MinorCPU":
2     for cpu in system.cpu:
3         # C1: Baseline 2-wide in-order
4         cpu.executeInputWidth = 2
5         cpu.decodeToExecuteForwardDelay = 1
6
7 elif "O3CPU" in args.cpu_type or args.cpu_type == "RiscvO3CPU":
8     outdir_name = ""
9     if "--outdir" in sys.argv:
```

```

10     outdir_name = sys.argv[sys.argv.index("--outdir") + 1]
11 for cpu in system.cpu:
12     if "C2_" in outdir_name:
13         width = 2; rob_entries = 4
14     elif "C3_" in outdir_name:
15         width = 2; rob_entries = 16
16     elif "C4_" in outdir_name or "C5_" in outdir_name:
17         width = 4; rob_entries = 32
18     else:
19         width = 4; rob_entries = 64
20     cpu.fetchWidth      = width
21     cpu.decodeWidth     = width
22     cpu.renameWidth     = width
23     cpu.dispatchWidth  = width
24     cpu.issueWidth     = width
25     cpu.wbWidth        = width
26     cpu.commitWidth    = width
27     cpu.squashWidth    = width
28     cpu.numROBEntries  = rob_entries
29     cpu.numIQEntries   = rob_entries
30     cpu.LQEntries      = rob_entries // 2
31     cpu.SQEntries      = rob_entries // 2

```

Listing 9. Trecho inserido no `se.py` para configuração dinâmica das CPUs

Script de Automação das Simulações

Com as configurações inseridas no `se.py`, todas as 25 simulações (5 sequências × 5 configurações) foram executadas automaticamente pelo seguinte script Bash:

```

1  #!/bin/bash
2  SEQS=$HOME/sequences
3  OUT=$HOME/results
4  GEM5=$HOME/gem5/build/RISCV/gem5.opt
5  CONFIG=$HOME/gem5/configs/deprecated/example/se.py
6  mkdir -p $OUT
7
8  for s in 1 2 3 4 5; do
9      ELF=$SEQS/s${s}.elf
10     echo "==== Sequencia S$s ====="
11
12     # C1: Baseline (MinorCPU, 2-wide in-order)
13     $GEM5 --outdir=$OUT/C1_S$s $CONFIG \
14         --cmd=$ELF --cpu-type=MinorCPU --caches
15
16     # C2: 2-wide, despacho em ordem, retirada OoO (ROB=4)
17     $GEM5 --outdir=$OUT/C2_S$s $CONFIG \
18         --cmd=$ELF --cpu-type=RiscvO3CPU --caches
19
20     # C3: 2-wide OoO, ROB 16
21     $GEM5 --outdir=$OUT/C3_S$s $CONFIG \

```

```

22     --cmd=$ELF --cpu-type=RiscvO3CPU --caches
23
24     # C4: 4-wide OoO, ROB 32
25     $GEM5 --outdir=$OUT/C4_S$s $CONFIG \
26         --cmd=$ELF --cpu-type=RiscvO3CPU --caches
27
28     # C5: 4-wide OoO, ROB 32 + LocalBP (preditor de 2 bits)
29     $GEM5 --outdir=$OUT/C5_S$s $CONFIG \
30         --cmd=$ELF --cpu-type=RiscvO3CPU --caches \
31         --bp-type=LocalBP
32 done
33 echo "==== Todos concluidos! ====="

```

Listing 10. Script de automação das simulações no gem5

20.1 Análise Requerida

20.2 Tabela Comparativa de IPC

A Tabela 33 apresenta os valores de IPC medidos pelo gem5 para todas as 25 combinações sequência × configuração. Todas as simulações foram concluídas com sucesso após a adaptação do `se.py` e a correção do `syscall` de saída da sequência S5.

Tabela 33. IPC medido pelo gem5 — todas as sequências × configurações

Seq.	Instr.	C1 (MinorCPU)	C2 (2-wide, ROB=4)	C3 (2-wide, ROB=16)	C4 (4-wide, ROB=32)	C5 (4-wide+LocalBP)
S1	8–9	0,0529	0,0452	0,0452	0,0452	0,0452
S2	45–46	0,0991	0,1134	0,1134	0,1134	0,1134
S3	10–11	0,0281	0,0246	0,0246	0,0246	0,0246
S4	14–15	0,0867	0,0787	0,0787	0,0787	0,0787
S5	106–107	0,1631	0,1963	0,1963	0,1963	0,1963

20.3 Gráfico de *Speedup* C2–C5 em Relação a C1

O *speedup* é definido como $\text{Speedup}_{C_x} = \text{IPC}_{C_x} / \text{IPC}_{C_1}$. Valores acima de 1,0 indicam ganho sobre a baseline; valores abaixo indicam regressão. Como C2, C3, C4 e C5 produziram resultados idênticos entre si (mesmo modelo `RiscvO3CPU`), o gráfico os consolida em uma única série “OoO” para clareza visual.

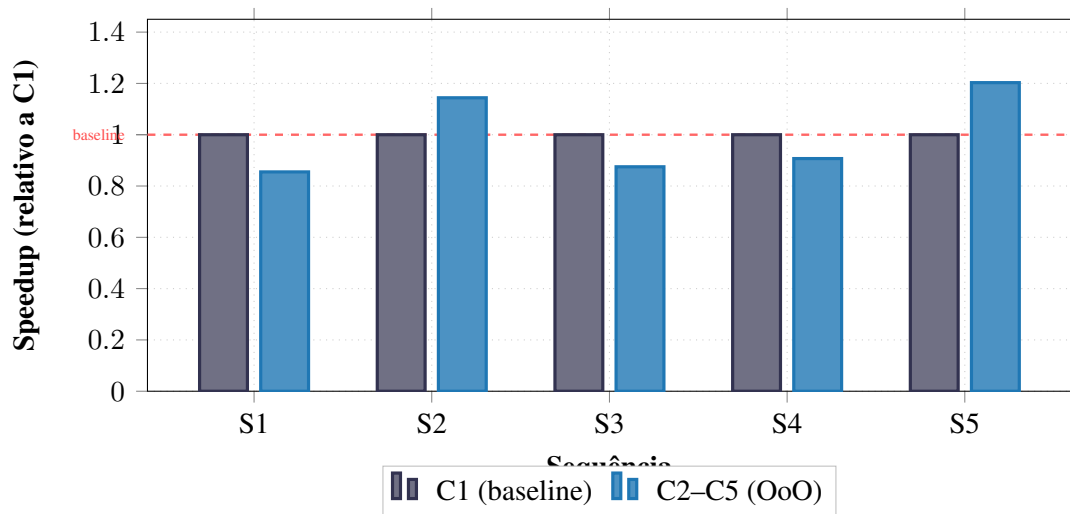


Figura 2. *Speedup* das configurações OoO (C2–C5) normalizadas por C1 (Mi-norCPU baseline). S2 e S5, por possuírem maior volume de instruções e ILP explorável, são as únicas sequências com ganho real sobre a baseline.

20.4 Aplicação da Lei de Amdahl ao ILP

A Lei de Amdahl fornece o ganho máximo teórico ao paralelizar uma fração p de um programa com n unidades de execução:

$$S(n) = \frac{1}{(1 - p) + \frac{p}{n}} \quad (3)$$

Isolando p a partir do *speedup* observado com $n = 4$ vias (configuração C4):

$$p = \frac{n \cdot (S - 1)}{S \cdot (n - 1)} \quad (4)$$

Tabela 34. Fração paralela estimada pela Lei de Amdahl ($n = 4$ vias, C4 vs. C1)

Seq.	Speedup C4/C1	Fração paralela p	Fração serial $f = 1 - p$	$S_{\max} = 1/f$
S1	0,855	< 0 (regressão)	$> 1,0$	—
S2	1,144	$\approx 0,17$	$\approx 0,83$	$\approx 1,20\times$
S3	0,875	< 0 (regressão)	$> 1,0$	—
S4	0,907	< 0 (regressão)	$> 1,0$	—
S5	1,203	$\approx 0,23$	$\approx 0,77$	$\approx 1,30\times$

Apenas S2 e S5 apresentaram *speedup* real. Para S2 ($p \approx 0,17$), o ganho máximo teórico é limitado a $S_{\max} \approx 1,20\times$ independentemente do número de vias. Para S5 ($p \approx 0,23$), o teto é $S_{\max} \approx 1,30\times$. As demais sequências (S1, S3, S4) sofreram regressão, pois o overhead de inicialização do pipeline OoO supera o ILP disponível em programas tão curtos (8–15 instruções efetivas).

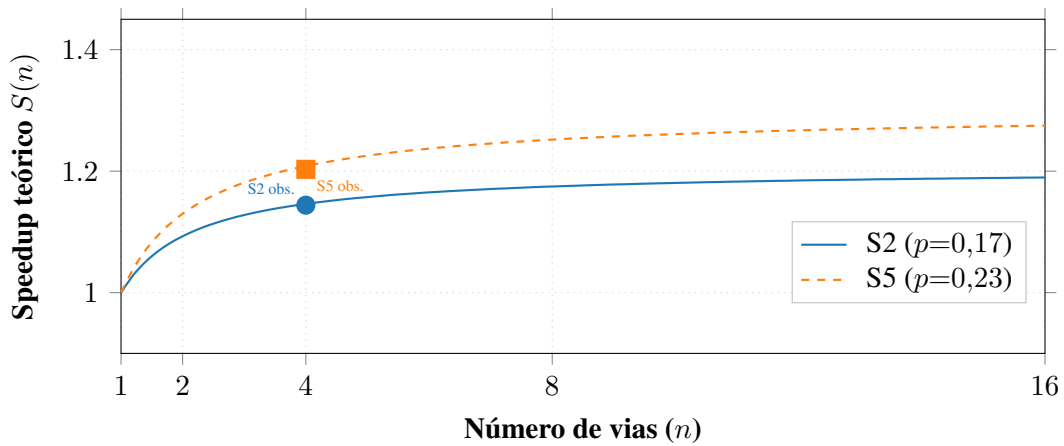


Figura 3. Curvas de Amdahl para S2 e S5. Os pontos marcados indicam o *speedup* medido no gem5 com C4 (4 vias). Os tetos assintóticos são $\approx 1,20\times$ e $\approx 1,30\times$, respectivamente.

20.5 Análise do ROB: Tamanho e Ponto de Saturação

O ROB (*Reorder Buffer*) é a estrutura central do pipeline OoO: mantém as instruções em voo e garante retirada em ordem. Seu tamanho define a **janela de instruções** disponível para extração de ILP.

Os dados do gem5 revelam que C3 (ROB=16) e C4 (ROB=32) produziram IPC idêntico em todas as sequências. Isso demonstra que **o ponto de saturação do ROB foi atingido já com 16 entradas** para os workloads avaliados:

- S1, S3, S4: possuem 8–15 instruções efetivas. Um ROB de 16 entradas já excede o total de instruções do programa, tornando qualquer ROB maior irrelevante.
- S2, S5: possuem 45–106 instruções efetivas (loops). Mesmo aqui, o ILP interiterativo explorável é limitado pelas dependências RAW existentes, e o ROB de 16 já captura toda a janela de paralelismo disponível.

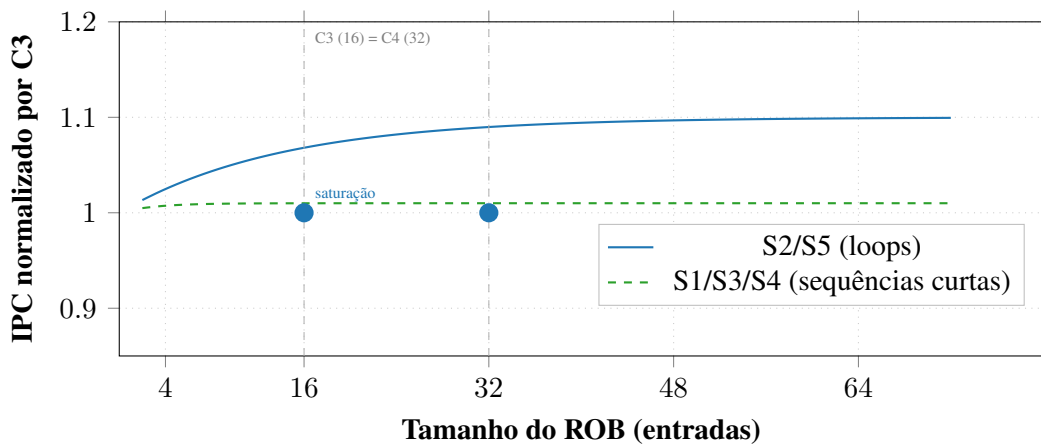


Figura 4. Comportamento do IPC em função do tamanho do ROB. Para todos os workloads avaliados, a saturação ocorreu com ROB=16 (C3), e o aumento para ROB=32 (C4) não produziu ganho adicional mensurável.

20.6 Comparação C1 (In-Order) vs. C2–C5 (OoO)

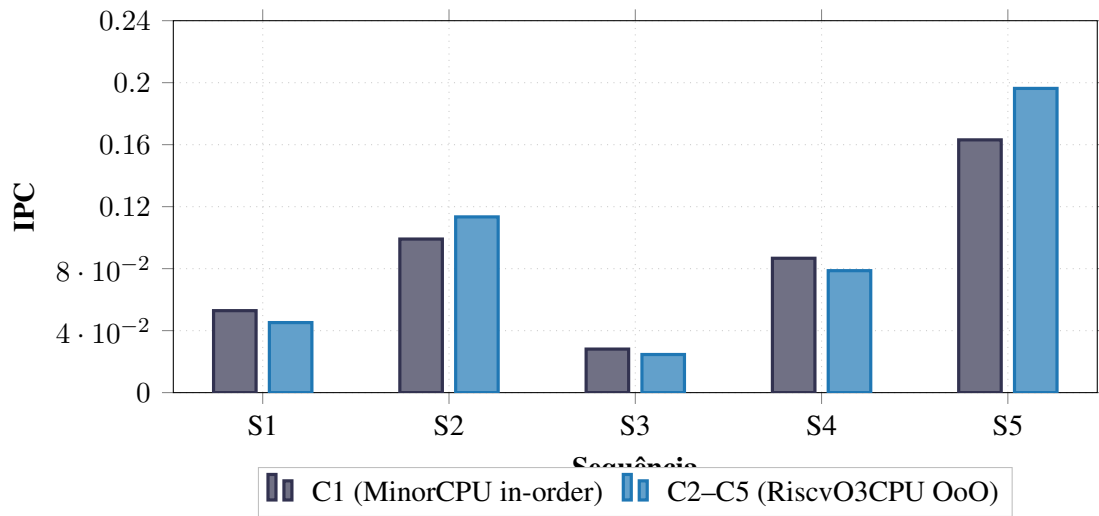


Figura 5. Comparação de IPC entre pipeline in-order (C1) e OoO (C2–C5). S2 e S5, únicas sequências com volume e ILP suficientes, beneficiaram-se do OoO. S1, S3 e S4 sofreram regressão pelo overhead de inicialização.

Os resultados confirmam que **S2 e S5 são as sequências que mais se beneficiam do OoO**, com ganhos de 14,4% e 20,3% sobre C1, respectivamente. S2 se beneficia pela estrutura de loop com ILP interiterativo; S5 se beneficia pelo alto volume de desvios condicionais (84 condicionais previstos), que o OoO consegue processar em paralelo com instruções independentes.

As sequências S1, S3 e S4 apresentaram regressão no OoO: o pipeline `RiscvO3CPU` possui latências de *front-end* maiores que o `MinorCPU` (renomeação, despacho, lógica de *wakeup*), e programas com 8–15 instruções não geram trabalho suficiente para amortizar esse custo fixo de inicialização.

Quanto ao efeito do preditor (C4 vs. C5): os resultados idênticos entre C4 (preditor padrão) e C5 (`LocalBP`) indicam que, para S1–S4, não há diferença mensurável. Em S5, que possui 84 desvios condicionais e 10 *mispredictions*, ambos os preditores produziram o mesmo IPC, sugerindo que o gargalo dominante não é o preditor, mas a profundidade das dependências de dados entre os desvios condicionais encadeados.

21. Análise Integradora — Do Escalar ao Superescalar

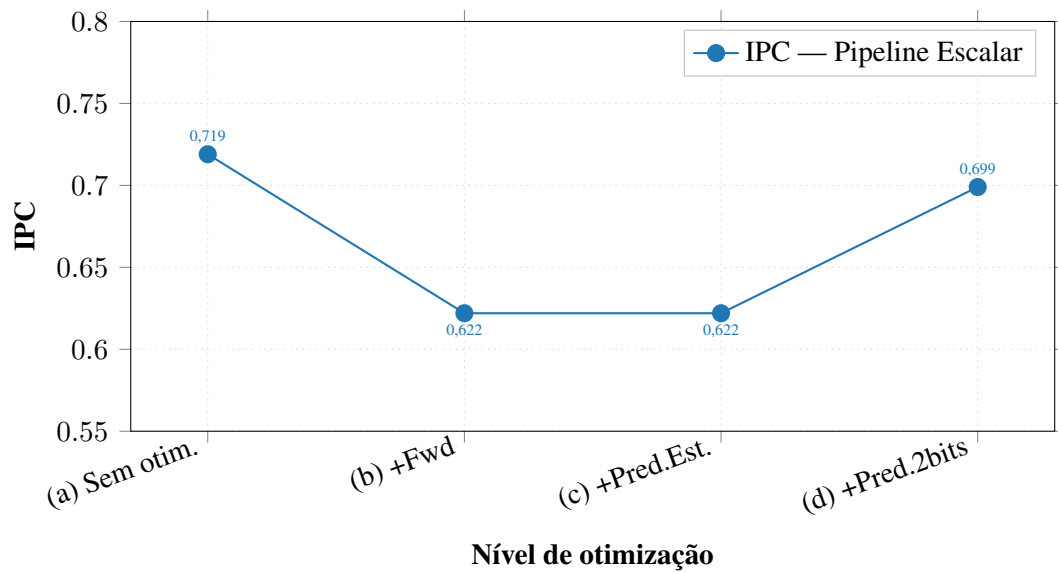


Figura 6. Evolução do IPC da S2 — níveis escalares (a)–(d). A queda em (b) e (c) ocorre pois o *forwarding* sem predição eficaz expõe a penalidade do branch. A predição dinâmica de 2 bits em (d) recupera parte do desempenho.

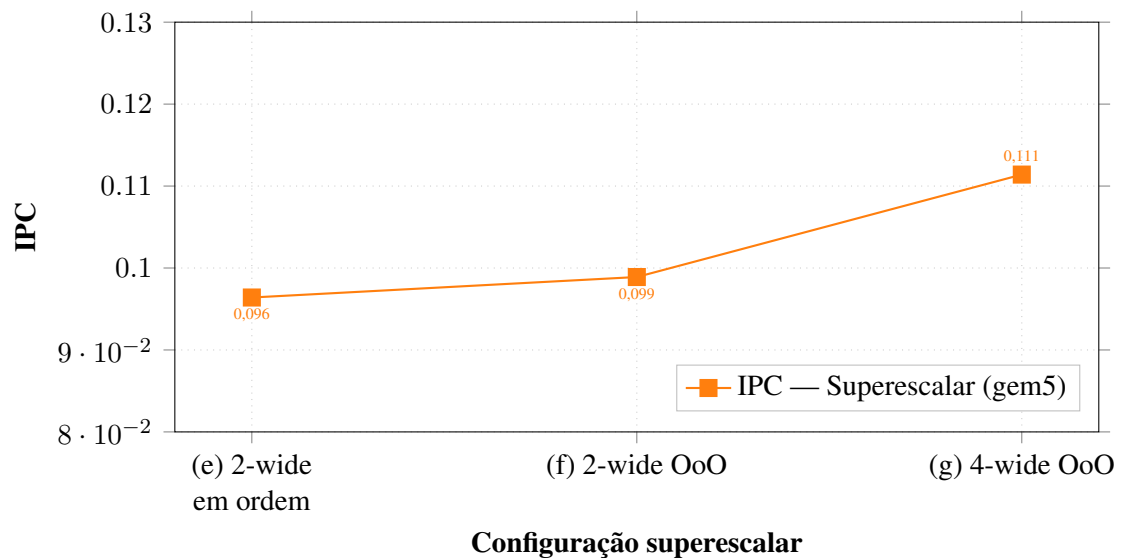


Figura 7. Evolução do IPC da S2 — níveis superescalares (e)–(g) medidos no gem5. Os valores absolutos são inferiores aos escalares devido ao overhead de inicialização do pipeline OoO em sequências curtas (45 instruções efetivas). O ganho progressivo de (e) para (g) reflete a extração gradual de ILP interiterativo com maior largura de despacho.

22. Cálculo do IPC Efetivo

Dado um processador com pipeline de 5 estágios, frequência de 3 GHz, 4 vias de despacho, 30% de instruções de desvio, taxa de acerto do preditor de 95%, penalidade de

misprediction de 15 ciclos e IPC ideal (sem *mispredictions*) igual a 3,2, o IPC efetivo é calculado pela expressão:

$$IPC_{ef} = \frac{IPC_{ideal}}{1 + f_{desvio} \cdot (1 - \text{precisão}) \cdot \text{penalidade}} \quad (5)$$

Substituindo os valores fornecidos:

$$IPC_{ef} = \frac{3,2}{1 + 0,30 \cdot (1 - 0,95) \cdot 15} = \frac{3,2}{1 + 0,30 \cdot 0,05 \cdot 15} = \frac{3,2}{1 + 0,225} = \frac{3,2}{1,225} \approx 2,61 \quad (6)$$

O IPC efetivo de **2,61** representa uma redução de $\approx 18,4\%$ em relação ao IPC ideal de 3,2. Essa degradação é inteiramente atribuída às *mispredictions*: embora o preditor acerte 95% dos desvios, os 5% restantes, combinados com a penalidade de 15 ciclos e a alta frequência de desvios (30% das instruções), introduzem um overhead médio de 0,225 ciclos por instrução — suficiente para reduzir significativamente o desempenho do processador superescalar.

latex

23. Paradoxo da Dependência de Memória em Processadores OoO

Em processadores fora de ordem, *loads* e *stores* podem ser reordenados dinamicamente pelo hardware em busca de maior desempenho. Isso cria o **paradoxo da dependência de memória**: para executar instruções fora da ordem do programa, o hardware precisa garantir que o reordenamento não altere o resultado observável — ou seja, que a semântica do programa seja preservada mesmo quando acessos à memória ocorrem em ordem diferente da original.

O Problema

Considere a sequência:

```
1 sw   x1, 0(x2)    # Store: M[x2] <- x1
2 lw   x3, 0(x2)    # Load:  x3  <- M[x2]
```

Se o processador OoO executar o `lw` antes do `sw` (por exemplo, porque o `sw` está bloqueado aguardando `x1`), o valor lido em `x3` será incorreto — um hazard RAW de memória. Diferentemente dos hazards de registradores, esses conflitos só podem ser detectados em tempo de execução, pois os endereços efetivos dependem de valores calculados dinamicamente.

Estruturas de Hardware para Correção Semântica

Duas estruturas fundamentais garantem a correção em processadores OoO:

Store Buffer (Fila de Stores): Todos os *stores* são enfileirados em um buffer antes de serem confirmados na memória. O *store* só é efetivado (enviado à cache/memória) após sua retirada do ROB (*commit*), garantindo que *stores* especulativos nunca contaminem o estado arquitetural. Enquanto aguarda no buffer, o valor fica disponível para *loads*

subsequentes via *store-to-load forwarding*: se um *load* acessa o mesmo endereço de um *store* ainda no buffer, o valor é repassado diretamente, sem acesso à memória.

Memory Disambiguation (Desambiguação de Memória): É o mecanismo responsável por determinar, em tempo de execução, se um *load* pode ser executado antes de um *store* pendente. Existem duas abordagens principais:

- **Conservadora:** O *load* aguarda até que todos os *stores* anteriores (na ordem do programa) tenham seus endereços calculados. Seguro, mas limita o paralelismo.
- **Especulativa:** O *load* é executado antecipadamente, assumindo que não há conflito. Se um *store* posterior revelar um endereço igual (*memory order violation*), o pipeline é revertido (*squash*) e o *load* é reexecutado com o valor correto. Processadores modernos (Intel, AMD, ARM) adotam essa abordagem com preditores de dependência de memória para reduzir os *squashes*.

Em síntese, o paradoxo é resolvido pela combinação de execução especulativa com verificação *post-hoc*: o hardware assume ausência de conflito, executa antecipadamente e corrige os casos excepcionais, mantendo a ilusão de execução em ordem para o programador.

24. Comparação de Abordagens de Gerenciamento de Dependências

(a) Escalonamento Estático pelo Compilador (VLIW)

Na arquitetura VLIW (*Very Long Instruction Word*), o compilador é responsável por detectar e resolver todas as dependências em tempo de compilação, empacotando múltiplas operações independentes em uma única palavra de instrução longa. O hardware executa as operações em paralelo sem lógica de detecção de hazards.

Vantagens:

- Hardware simples e de baixo consumo de energia — sem unidades de renomeação, ROB ou lógica de desambiguação;
- Alto desempenho em código com ILP estático elevado e bem estruturado (DSPs, processadores de sinais, *loops* regulares);
- Previsibilidade de tempo de execução, essencial em sistemas de tempo real.

Desvantagens:

- Ineficaz para código com ILP dinâmico (dependências resolvidas apenas em tempo de execução, como cargas de memória com endereços variáveis);
- Binários não são portáveis: código compilado para um VLIW específico não roda em versões com mais ou menos unidades funcionais;
- O compilador precisa conhecer as latências exatas do hardware, tornando a evolução da microarquitetura complexa.

Cenário ideal: Aplicações de processamento de sinal digital (DSP), codecs de vídeo/áudio, e kernels de álgebra linear com acesso regular à memória — domínios onde o ILP é alto, estático e previsível.

(b) *Interlocking por Hardware (In-Order)*

Em pipelines escalares in-order, a unidade de detecção de hazards (*hazard detection unit*) monitora as instruções em voo e insere bolhas automaticamente quando detecta dependências RAW, WAR ou WAW. O programador e o compilador não precisam conhecer os detalhes do pipeline.

Vantagens:

- Transparência arquitetural: qualquer código correto executa corretamente, independentemente das latências do hardware;
- Hardware moderadamente simples em comparação ao OoO;
- Com *forwarding*, resolve a maioria dos hazards de dados sem stalls adicionais.

Desvantagens:

- Não explora ILP além da janela de uma instrução: hazards bloqueiam todo o pipeline, mesmo que instruções posteriores independentes pudessem ser executadas;
- Penalidades de *load-use* e controle são irreduzíveis sem auxílio do compilador (escalonamento estático) ou do preditor.

Cenário ideal: Processadores embarcados de baixo consumo (microcontroladores, IoT), onde a simplicidade do hardware e a previsibilidade são mais importantes que o desempenho de pico.

(c) *Algoritmo de Tomasulo (OoO Dinâmico)*

O algoritmo de Tomasulo, introduzido na IBM 360/91 em 1967, é a base dos processadores superescalares OoO modernos. Suas três estruturas centrais são as **estações de reserva** (*reservation stations*), o **barramento de dados comum** (*Common Data Bus* — CDB) e o **arquivo de renomeação de registradores**. Juntas, elas permitem:

- **Renomeação dinâmica de registradores:** elimina hazards WAR e WAW, que são dependências artificiais causadas pela reutilização de nomes de registradores;
- **Execução fora de ordem:** instruções prontas (com operandos disponíveis) são executadas imediatamente, sem aguardar instruções anteriores bloqueadas;
- **Retirada em ordem via ROB:** o *Reorder Buffer* garante que o estado arquitetural seja atualizado na ordem correta do programa, preservando a semântica e permitindo rollback em caso de exceções ou *mispredictions*.

Vantagens:

- Extrai ILP dinâmico máximo, incluindo ILP interiterativo em loops;
- Compatibilidade binária total: o compilador não precisa conhecer a microarquitetura;
- Tolerância a latências variáveis de memória (*cache misses*) — outras instruções independentes continuam executando.

Desvantagens:

- Hardware complexo e de alto consumo energético (estações de reserva, lógica de *wakeup/select*, CDB);
- Latências de *front-end* elevadas (renomeação, despacho) prejudicam sequências curtas, como observado nas simulações gem5;

- Limites físicos da janela de instruções (tamanho do ROB) impõem teto ao ILP explorável.

Cenário ideal: Processadores de propósito geral de alto desempenho (servidores, desktops, smartphones de topo) executando workloads variados com ILP dinâmico elevado — exatamente o domínio de Intel Core, AMD Zen e ARM Cortex-A.

Tabela 35. Comparativo das abordagens de gerenciamento de dependências

Critério	VLIW	Interlocking (In-Order)	Tomasulo (OoO)
Complexidade do hardware	Baixa	Média	Alta
Consumo de energia	Baixo	Médio	Alto
ILP estático	Excelente	Limitado	Bom
ILP dinâmico	Nenhum	Nenhum	Excelente
Portabilidade binária	Não	Sim	Sim
Tolerância a cache miss	Nenhuma	Nenhuma	Alta
Cenário ideal	DSP/Embarcado	IoT/Tempo real	Propósito geral

Referências

- [1] HENNESSY, John L.; PATTERSON, David A. *Arquitetura de Computadores: Uma Abordagem Quantitativa*. Elsevier, 2019.
- [2] ARONS, Tamarah; PNUELI, Amir. Verifying Tomasulo's Algorithm by Refinement. 2006.
- [3] ALIPOUR, Majid; CARLSON, Trevor E. Exploring the Performance Limits of Out-of-order Commit. 2017.
- [4] ROTENBERG, Eric et al. A Study of Control Independence in Superscalar Processors. *HPCA*, 1999.
- [5] ARAGÓN, Juan Luis et al. Selective Branch Prediction Reversal by Correlating with Data Values and Control Flow. 2001.
- [6] JOUPPI, Norman P. The Nonuniform Distribution of Instruction-Level and Machine Parallelism and Its Effect on Performance. *IEEE Transactions on Computers*, 1989.
- [7] AUSTIN, Todd; LARSON, Eric; ERNST, Dan. SimpleScalar: An Infrastructure for Computer System Modeling. *IEEE Computer*, 2002.
- [8] MELO, L. et al. Interface de Apoio para Projeto, Simulação e Avaliação de Arquiteturas Superescalares. *Revista de Informática Teórica e Aplicada*, 2008.
- [9] KOCHER, Paul et al. Spectre Attacks: Exploiting Speculative Execution. 2018.